

Puls-Events — Étude de design d'un MVP RAG cloud-native

Rapport de gestion de projet et d'architecture — Data Engineer RNCP Niveau 7

Behram Korkut

Juin 2026

Table des matières

Introduction	2
Contexte du projet	2
Objectif et nature de la mission	2
Problématique	2
Démarche et structure du rapport	3
Périmètre et limites assumées	3
Section 1 — Analyse et synthèse des besoins	3
Section 2 — Plan de projet	13
Section 3 — Macro backlog des fonctionnalités	20
Section 4 — Architecture technique	29
4.2 Choix des composants applicatifs	32
4.3 — Choix des composants applicatifs (Groupe 2 : composants de support)	39
4.4 — Schéma d'architecture (description textuelle) schéma en quatre vues	47
Sous-section 4.5 — Stratégies transverses	51
4.5 Stratégies transverses	51
Section 5 : Estimation des coûts BUILD et OPEX	60
5.1 Hypothèses de charge — trois scénarios	60
5.2 Coûts BUILD — investissement initial sur 12 semaines	61
5.3 Coûts OPEX — exploitation récurrente par scénario	63
5.4 Synthèse globale sur 12 mois	65
5.5 Leviers d'optimisation budgétaire	66
5.6 Annexe : disposition Excel / CSV	67
Conclusion du rapport	67
Synthèse	67
Apprentissages structurants	68
Perspectives post-MVP	68
Position personnelle	69
Annexes	69
A1. Portfolio professionnel	69
A2. Repositories GitHub clés	69
A3. Schémas d'architecture	70
A4. Fichiers de données	70
A5. Journal de bord du projet	70
A6. Sources et références bibliographiques	71
A7. Outils utilisés pour la production du rapport	72

A8. Glossaire des acronymes	72
---------------------------------------	----

Introduction

Contexte du projet

Le marché de la découverte culturelle vit une transformation profonde sous l'effet conjugué de la maturation des modèles de langage (LLM) et de l'émergence des architectures **RAG** (Retrieval-Augmented Generation). En quelques années, les utilisateurs ont basculé d'une logique de recherche par mots-clés vers des attentes conversationnelles : ils veulent **poser des questions en langage naturel** et obtenir des recommandations personnalisées, contextuelles, et fiables.

C'est dans ce contexte que **Puls-Events**, plateforme web spécialisée dans la découverte d'événements culturels en temps réel, a lancé une démarche d'innovation visant à intégrer un chatbot intelligent à son offre. Cette démarche a démarré par un **POC** (Proof of Concept) développé par mes soins pendant ma formation, démontrant la faisabilité technique d'un moteur de recherche sémantique basé sur OpenAgenda, Mistral AI, FAISS et LangChain. Évalué sur 10 questions annotées avec un recall de 95% et un score global de 3,9/5, ce POC a convaincu les équipes produit et marketing.

La présente mission consiste à **transformer ce POC en MVP scalable** prêt à être déployé en production, et à intégrer quatre nouvelles capacités identifiées comme différenciantes par l'équipe : mémoire conversationnelle, contexte géographique optimisé, recherche web temps réel, et monitoring de performance.

Objectif et nature de la mission

Contrairement à la mission POC qui évaluait des compétences d'**exécution technique** (savoir construire), la présente mission évalue des compétences de **conception et de pilotage** (savoir décider, planifier, justifier, anticiper). Le livrable attendu n'est pas un produit fonctionnel mais un **rapport de gestion de projet complet** permettant à une équipe pluridisciplinaire de réaliser le MVP dans les délais, le budget, et le niveau de qualité attendus.

Cette bascule de posture — d'exécutant à **réfèrent technique** — constitue le fil rouge méthodologique du présent document. Elle mobilise les six compétences évaluées par le titre RNCP de niveau 7 visé : collecter et analyser les besoins, planifier et suivre le projet, contrôler les délais et coûts, apporter un appui stratégique, identifier de nouvelles opportunités, et conduire les échanges entre parties prenantes.

Problématique

Comment concevoir et planifier un MVP RAG cloud-native qui soit à la fois **scalable** (capable d'absorber une croissance d'usage de 50 à 50 000 utilisateurs), **maîtrisé** (budget, délais, conformité RGPD), et **réversible** (capable d'évoluer avec un écosystème IA en mutation rapide), tout en restant **réalisable** par une équipe restreinte dans un horizon de 12 semaines ?

Démarche et structure du rapport

Le rapport suit la structure du template fourni par le responsable technique, organisée en six sections complémentaires :

- **Section 1** présente l'analyse et la synthèse des besoins, le décodage du brief reçu, les contraintes techniques, les personas utilisateurs et la composition de l'équipe projet.
- **Section 2** définit le plan de projet, la méthodologie retenue (Scrumban hybride), les jalons stratégiques et l'échéancier sur 12 semaines.
- **Section 3** présente le macro backlog de 36 fonctionnalités priorisées en MoSCoW, estimées en story points Fibonacci, avec cartographie des risques.
- **Section 4** détaille l'architecture technique : choix du cloud provider et des composants applicatifs justifiés par une veille structurée, schéma d'architecture en quatre vues, stratégies transverses (déploiement, modularité, monitoring, sécurité).
- **Section 5** estime les coûts BUILD (investissement initial) et OPEX (exploitation récurrente) selon trois scénarios de charge, avec leviers d'optimisation chiffrés.
- **Section 6** dresse le bilan du chemin parcouru du POC au MVP, justifie les choix structurants et ouvre sur les perspectives post-MVP.

Périmètre et limites assumées

Trois limites du présent rapport méritent d'être posées explicitement :

D'abord, la définition des **personas utilisateurs** repose sur une démarche de recherche secondaire (analyse concurrentielle, études sectorielles, données POC) en l'absence d'interviews primaires. Ces personas sont **étiquetés comme hypothèses de travail** et seront validés en sprint Discovery du projet réel.

Ensuite, l'**estimation des coûts** s'appuie sur des hypothèses unitaires de charge (volume de requêtes, taille de tokens, TJM marché) qui sont documentées et seront réajustées trimestriellement sur la base des observations Langfuse et Snowflake en production.

Enfin, le rapport traite l'**investissement** mais pas le **retour sur investissement** : le calcul de ROI nécessite un business plan revenu (abonnements, freemium, partenariats) qui relève de l'équipe produit et dépasse le périmètre du design technique.

Le portfolio professionnel complet, qui contextualise les compétences mises en œuvre dans ce rapport, est accessible en annexe sous forme d'un lien unique vers une page Notion curatée. Le journal de bord détaillé de la démarche de conception est également disponible dans le repository GitHub associé à ce rapport.

Section 1 — Analyse et synthèse des besoins

1.1 Synthèse du contexte

Puls-Events est une plateforme web de découverte d'événements culturels en temps réel

Un POC a démontré la faisabilité d'un moteur de recherche sémantique basé sur un chatbot (Augmented Generation), exploitant une base vectorielle FAISS, le modèle d'embeddings

Le présent projet vise à transformer ce POC en MVP scalable capable d'être déployé en native, d'absorber une charge utilisateur croissante, et d'intégrer quatre nouvelles

1.2 Objectifs métiers

D	Objectif	Indicateur de succès
OM1	Augmenter la rétention utilisateur via la personnalisation	+30% de retour utilisateur sous 3 mois post-MVP
OM2	Élargir la couverture géographique de l'IDF à la France entière	100% des régions métropolitaines couvertes au lancement
OM3	Réduire le temps de découverte d'un événement pertinent	< 30s entre la question et une recommandation utilisable
OM4	Permettre des recommandations d'actualité (au-delà du dataset)	Capacité à répondre sur des événements non indexés via recherche web
OM5	Mesurer la satisfaction et piloter l'amélioration continue	Dashboard monitoring opérationnel dès le lancement

1.3 Décodage des besoins de Jérémie

Besoin exprimé par Jérémie	Type	Interprétation technique	Décision de design
"Mémoire conversationnelle"	Fonctionnel critique	Persistance des échanges par utilisateur, exploitation de l'historique pour les requêtes suivantes	LangGraph Checkpointer + DynamoDB (NoSQL serverless, low-latency, free tier AWS généreux)

Besoin exprimé par Jérémie	Type	Interprétation technique	Décision de design
“Contexte géographique optimisé”	Fonction critique	Géolocalisation utilisateur + tri/filtrage des résultats par proximité	Géocodage via API BAN (gouv.fr, souverain, gratuit), enrichissement metadata Qdrant (lat/lng), filtre géographique + scoring par distance
“Recherche web en temps réel”	Fonction critique	Capacité de l’agent à interroger le web quand le RAG est insuffisant	Tavily Search API (standard 2025-2026 pour search-for-LLM, free tier 1000 req/mois) appelé comme un tool LangGraph conditionnel
“Smolagents de Hugging Face”	Suggestion à challenger	Framework agent code-acting (l’agent écrit et exécute du Python)	Non retenu : (1) écosystème LangGraph déjà en place, (2) risque sécurité prod (exécution code), (3) meilleure observabilité avec LangGraph + Langfuse. Documenté comme veille comparative.
“Monitoring de performance”	Non-fonction critique	Mesure technique (latence, coût, erreurs) + métier (satisfaction, qualité réponses)	Langfuse pour l’observabilité LLM (traces, coûts, feedback) + CloudWatch pour l’infra
“Scalable”	Non-fonction critique	Capacité à monter en charge sans refonte	Architecture conteneurisée stateless sur ECS Fargate, auto-scaling, base vectorielle managée

Besoin exprimé par Jérémy	Type	Interprétation technique	Décision de design
“Robuste”	Non-fonctionnel	Tolérance aux pannes, qualité du code	Multi-AZ, healthchecks, tests automatisés, CI/CD GitHub Actions
“Innovant”	Soft / marketing	Démontrer l’usage de techno de pointe	Stack 2026 : LangGraph agentic + Qdrant + Mistral + Langfuse + Snowflake

1.4 Contraintes techniques

Type	Contrainte	Impact sur le design
Budget	Maîtrise des coûts (alternance, MVP early-stage)	AWS Free Tier + Snowflake Free Trial + tiers gratuits des SaaS LLM
Souveraineté	Données utilisateurs français/européens	Région AWS eu-west-3 (Paris), géocodage BAN gouv.fr, Mistral AI (FR)
Performance	Latence end-to-end < 5s perçue par l'utilisateur	Cache embeddings, streaming des réponses LLM, Qdrant Cloud co-localisé
Conformité	RGPD (données conversationnelles)	Anonymisation des logs analytiques Snowflake, consentement explicite, TTL sur historique
Scalabilité	Passer de 50 à 10 000 utilisateurs sans refonte	Stateless app + bases managées + auto-scaling
Maintenance	Équipe réduite (alternant + tech lead)	Services managés > self-hosted, IaC (Terraform), CI/CD automatisé

1.5 Démarche de définition des utilisateurs cibles

En phase de cadrage MVP, la définition des personas et des cas d’usage repose sur une démarche de **recherche secondaire** (desk research) à défaut d’une recherche primaire (interviews utilisateurs) qui sera menée en sprint 0 du projet. Cette approche, courante en lancement MVP, vise à dégager des hypothèses fortes à valider rapidement avec des utilisateurs réels avant l’implémentation des fonctionnalités les plus coûteuses.

Sources mobilisées

- **Analyse concurrentielle** : étude des solutions existantes sur le marché français de la découverte culturelle (Mapado, Sortir à Paris, OfficeDeTourisme.fr, Affluences, Cultival, Fnac Spectacles).
- **Études sectorielles** : rapport du Ministère de la Culture sur les pratiques culturelles des Français (2023), baromètre du numérique ARCEP-CREDOC, études Médiamétrie sur la consommation culturelle digitale.
- **Données d’usage du POC** : analyse des 30 requêtes annotées de l’évaluation du POC (typologie des questions, intentions exprimées, formulations naturelles, taux de réécriture de requête à 46.7%).
- **Données OpenAgenda** : analyse de la répartition des 5 000 événements indexés (catégories dominantes, distribution géographique sur 33 régions et 1 945 villes, profils d’organiseurs).
- **Retours informels** des équipes produit et marketing de Puls-Events (mentionnés dans la communication initiale du responsable technique).

Niveau de maturité actuel

⚠ **Niveau 1 — Personas hypothétiques.** Une démarche de validation primaire (interviews qualitatives de 8 à 12 utilisateurs cibles) est planifiée en **Jalon J1 — Discovery** du plan projet (cf. section 2). Les personas et cas d’usage présentés ci-dessous constituent donc une **base de travail à raffiner**, et non une vérité figée. Cette posture méthodologique honnête est volontaire : elle permet de séquencer l’investissement de validation utilisateur avant l’implémentation des briques les plus coûteuses du MVP (mémoire conversationnelle, recherche web temps réel).

1.6 Personas hypothétiques

Les personas ci-dessous sont des **hypothèses de travail** issues de la démarche desk research décrite en 1.5. Ils seront validés ou révisés à l’issue du sprint Discovery (J1).

Persona P1 — Léa, urbaine curieuse (*utilisateur primaire*)

Attribut	Valeur
Âge	28 ans
Localisation	Lyon, centre-ville
Profession	Chargée de communication
Pratique culturelle	2 à 3 sorties par mois (expos, concerts, théâtre)
Équipement	Smartphone principal, usage en mobilité

Attribut	Valeur
Frustration principale	Devoir scroller plusieurs sites pour trouver ce qui se passe ce week-end
Attente vis-à-vis du chatbot	Réponse rapide, personnalisée, sans avoir à formuler des critères précis
Cas d'usage type	« <i>Qu'est-ce qu'il y a comme expo gratuite à Lyon ce week-end ?</i> »

Persona P2 — Karim, parent organisateur (*utilisateur secondaire*)

Attribut	Valeur
Âge	42 ans
Localisation	Paris, 19 ^e arrondissement
Profession	Ingénieur, père de deux enfants (6 et 9 ans)
Pratique culturelle	Sorties familiales mercredi/samedi
Équipement	Mobile et desktop selon le contexte
Frustration principale	Difficulté à filtrer par tranche d'âge et accessibilité transport
Attente vis-à-vis du chatbot	Comprendre des critères complexes en langage naturel (âge enfants, mobilité)
Cas d'usage type	« <i>Un atelier créatif pour mes enfants de 6 et 9 ans, samedi matin, accessible en métro à Paris ?</i> »

Persona P3 — Sophie, voyageuse week-end (*utilisateur étendu*)

Attribut	Valeur
Âge	35 ans
Localisation	Paris, mais voyage 1 à 2 week-ends par mois en France
Profession	Consultante indépendante
Pratique culturelle	Découverte de la programmation locale lors de ses déplacements
Équipement	Mobile en mobilité, usage géolocalisé fort
Frustration principale	Méconnaissance des scènes culturelles locales hors Paris

Attribut	Valeur
Attente vis-à-vis du chatbot	Suggestions actualisées d'événements dans une ville non familière
Cas d'usage type	« Je suis à Bordeaux ce week-end, qu'est-ce qu'il faut absolument voir niveau concerts ? »

Ce persona P3 justifie spécifiquement le besoin de **recherche web temps réel** : les événements très récents ou de petite scène locale peuvent ne pas être indexés dans OpenAgenda au moment de la requête.

Persona P4 — Équipe produit Puls-Events (*utilisateur interne*)

Attribut	Valeur
Profil	Product Manager, Data Analyst, Growth Manager
Besoin principal	Comprendre l'usage du chatbot, identifier les requêtes sans réponse, mesurer la satisfaction
Attente vis-à-vis du système	Dashboard d'analytics conversationnels, alertes sur dérives qualité
Cas d'usage type	« Quelles sont les 20 requêtes les plus fréquentes auxquelles le chatbot a mal répondu cette semaine ? »

Ce persona interne justifie l'investissement dans la brique **Snowflake + Langfuse** pour le monitoring et l'analytics conversationnels. Il rappelle qu'un produit data n'est jamais destiné uniquement aux utilisateurs finaux : l'équipe produit est elle aussi un consommateur de la donnée.

1.7 Cas d'usage prioritaires et user stories

Les cas d'usage ci-dessous découlent directement des personas P1 à P4. Ils sont **priorisés selon la méthode MoSCoW** (Must / Should / Could / Won't) et formalisés en user stories Agile dans le macro backlog (cf. section 3).

ID	Cas d'usage	Persona	Briques mobilisées	Priorité MVP
UC1	Recherche conversationnelle simple en langage naturel	P1, P2, P3	RAG (Qdrant + Mistral)	Must-Have
UC2	Affinage par mémoire conversationnelle (« et en plus jazz ? »)	P1, P2	RAG + Checkpointer DynamoDB	Must-Have
UC3	Recommandation géolocalisée par proximité	P2, P3	RAG + filtre géo Qdrant + API BAN	Must-Have
UC4	Découverte d'événements très récents non indexés	P3	RAG + Tavily Search (fallback agentique)	Must-Have
UC5	Collecte de feedback utilisateur sur une réponse ((+)/((-))	Tous	Langfuse user feedback	Must-Have
UC6	Pilotage produit via dashboard analytics conversationnels	P4	Snowflake + dashboard BI	Should-Have
UC7	Réponse multilingue (FR/EN)	P3	Mistral multilingue	Should-Have
UC8	Authentification utilisateur et historique long-terme	P1, P2	Cognito + DynamoDB	Could-Have
UC9	Notifications push d'événements correspondant aux préférences	P1, P2	SNS + scheduler	Won't-Have (post-MVP)

ID	Cas d'usage	Persona	Briques mobilisées	Priorité MVP
UC10	Réservation directe depuis le chatbot	P1, P2	Intégrations billetterie	Won't-Have (post-MVP)

Exemple de user story formalisée (template Agile)

US-UC3.1 : *En tant que* Karim (parent organisateur), *je veux* pouvoir demander un atelier près de mon domicile sans préciser l'adresse à chaque requête, *afin de* gagner du temps et obtenir des suggestions adaptées à ma mobilité réelle.

Critères d'acceptation : - Le chatbot détecte qu'une question contient une intention géographique implicite (« près de chez moi », « pas loin »). - Si la localisation utilisateur est connue (session ou profil), elle est utilisée comme centre de recherche. - Les résultats sont triés par distance croissante. - La distance est affichée à l'utilisateur (« à 2,3 km de votre position »).

Le backlog complet et toutes les user stories sont détaillés en **section 3 (Macro backlog)**.

1.8 Composition de l'équipe projet

L'analyse des besoins révèle un projet **pluridisciplinaire** mobilisant des compétences techniques, produit, design, données et conformité. Bien que le présent rapport soit produit par un Data Engineer en alternance positionné en pilote technique, le déploiement effectif du MVP suppose l'implication des profils décrits ci-dessous.

Rôle	Responsabilité sur le projet	Charge estimée sur 12 semaines	Statut
Data Engineer (moi, pilote technique)	Conception et implémentation de la stack RAG, pipelines de données, déploiement cloud, monitoring technique, IaC	100 %	Interne
Tech Lead / Responsable technique	Validation des choix d'architecture, code review, arbitrages techniques, mentorat	20 %	Interne

Rôle	Responsabilité sur le projet	Charge estimée sur 12 semaines	Statut
Product Owner	Priorisation du backlog, validation des user stories, lien avec parties prenantes, animation sprint review	30 %	Interne
UX/UI Designer	Maquettes interface chatbot, parcours utilisateur, design system, tests utilisabilité	50 % sur 4 semaines	Freelance ou interne
DevOps / SRE (en appui)	Validation Terraform, sécurisation AWS, revue de la stratégie d'observabilité	10 %	Interne ou mutualisé
Data Analyst	Construction des dashboards Snowflake, analyse des logs conversationnels post-lancement	20 % à partir de la semaine 8	Interne
Responsable juridique / DPO	Validation RGPD (mémoire conversationnelle, anonymisation, consentement, durée de conservation)	3 à 5 jours répartis	Interne ou conseil externe
Marketing / Growth	Préparation du lancement, communication utilisateurs, suivi adoption	20 % à partir de la semaine 10	Interne

Lecture de ce tableau

Cette composition d'équipe a trois implications structurantes pour la suite du rapport :

1. **Coût humain** : la majeure partie du coût de build d'un MVP est constituée du temps humain plutôt que de l'infrastructure cloud. Ce point sera quantifié en

section 5 (Estimation des coûts).

2. **Besoin de coordination** : avec sept profils impliqués à temps partiel, la mise en place de **rituels Agile structurés** (daily, sprint planning, sprint review, rétro) devient indispensable. Ce point justifie le choix de méthodologie présenté en section 2.
3. **Gestion des dépendances externes** : le DPO et le Designer doivent être mobilisés en début de projet pour éviter de bloquer le développement en milieu de sprint. Ces dépendances seront matérialisées dans les jalons (section 2) et le backlog (section 3).

Note de positionnement : dans le cadre de ce rapport et de la soutenance, je me positionne en pilote technique du MVP. Les charges des autres profils sont mentionnées à titre indicatif et seront affinées avec le Product Owner lors du kick-off du projet.

Section 2 — Plan de projet

Cette section présente la **méthodologie de pilotage** retenue pour le MVP, les **jalons stratégiques** du projet, l'**échéancier** détaillé sous forme de Gantt, et les **livrables** attendus à chaque étape.

2.1 Méthodologie projet

Démarche de sélection Quatre approches méthodologiques ont été évaluées au regard du contexte spécifique du projet Puls-Events MVP : équipe restreinte (2 à 4 ETP effectifs sur le développement), incertitude produit forte (personas hypothétiques à valider), nécessité de démos régulières aux parties prenantes, et durée totale de 12 semaines.

Critère	Cycle en V	Scrum pur	Kanban	Scrumban (re)
Adapté à une équipe 2-5 personnes	✗	⚠	✓	✓
Adapté à l'incertitude produit MVP	✗	✓	✓	✓
Démos régulières aux parties prenantes	⚠	✓	✗	✓
Adaptabilité en cours de cycle	✗	⚠	✓	✓
Cérémonies dimensionnées équipe restreinte	✓	✗	✓	✓
Reconnaissance industrie / jury	✓	✓	✓	✓

Justification du choix : Scrumban **Scrumban** est une approche hybride combinant le cadre temporel et rituel de **Scrum** (sprints, sprint review, rétrospective) avec la flexibilité du tableau visuel **Kanban** et ses limites de tâches en cours (WIP limits). Cette combinaison a été retenue pour quatre raisons :

1. **Dimensionnement équipe** : avec 2 à 4 développeurs effectifs, les quatre cérémonies hebdomadaires d'un Scrum strict représenteraient un overhead disproportionné. Scrumban autorise un allègement contrôlé (par exemple, daily stand-up de 10 min au lieu de 15, rétrospective bi-sprint au lieu de chaque sprint).

2. **Réactivité aux apprentissages utilisateurs** : un MVP intègre par nature des retours qui peuvent invalider une user story en cours de sprint. Le tableau Kanban permet de réintroduire ces nouveaux éléments sans attendre la fin du sprint, ce que Scrum pur interdit.
3. **Démos parties prenantes** : les sprint reviews bi-hebdomadaires permettent de maintenir l'alignement avec l'équipe produit, marketing et le responsable technique (Jérémy), conformément au besoin de "rassurer les sponsors" implicite dans la communication initiale.
4. **Gestion des interruptions opérationnelles** : un projet data/IA en début de vie subit fréquemment des incidents (quotas API, dérive qualité modèle, coût LLM imprévu). La WIP limit du Kanban évite la dispersion en forçant à terminer ce qui est commencé avant de prendre du nouveau travail.

Cadre opérationnel retenu

Élément	Choix
Durée de sprint	2 semaines
Nombre de sprints sur le projet	6 sprints
Daily stand-up	10 min, asynchrone si nécessaire
Sprint Planning	1h en début de sprint
Sprint Review	45 min en fin de sprint, démo + feedback parties prenantes
Rétrospective	30 min en fin de sprint
Tableau Kanban	GitHub Projects (intégré au repo)
WIP limit colonne "En cours"	3 tâches maximum
Definition of Done	Code mergé sur main + tests passants + documentation à jour + déployé en environnement de staging

Évolution post-MVP Si le MVP réussit son passage à l'échelle (>1 000 utilisateurs actifs mensuels) et que l'équipe s'agrandit (>5 développeurs), une transition vers **Scrum complet** sera reconsidérée pour bénéficier de sa structure plus rigoureuse.

2.2 Jalons stratégiques

Le projet est structuré autour de **6 jalons** correspondant aux points de contrôle Go/No-Go significatifs. Ces jalons sont indépendants du rythme des sprints : un jalon peut être franchi en cours de sprint.

ID	Jalon	Semaine cible	Critère de franchissement
J0	Kick-off projet validé	S0 (avant démarrage)	Rapport de cadrage validé par Jérémie, équipe constituée, accès AWS/Snowflake/Mistral provisionnés
J1	Discovery utilisateur achevé	Fin S2	8 à 12 interviews utilisateurs menées, personas validés ou révisés, backlog réajusté
J2	Infrastructure cloud opérationnelle	Fin S4	Terraform appliqué sur AWS, Qdrant Cloud connecté, pipeline d'ingestion fonctionnel, environnement de staging accessible
J3	Cœur fonctionnel MVP livré	Fin S6	Mémoire conversationnelle + contexte géographique + recherche web opérationnels en staging, démontrés en sprint review
J4	Monitoring et qualité validés	Fin S8	Langfuse + dashboards Snowflake en production, taux de réponse pertinente > 80% sur jeu d'évaluation étendu (50 questions)

ID	Jalon	Semaine cible	Critère de franchissement
J5	MVP en production	Fin S10	Déploiement public, premiers utilisateurs réels, SLA disponibilité 99% sur 7 jours consécutifs
J6	Bilan et passage à l'échelle décidé	Fin S12	Rapport post-MVP avec métriques d'usage, recommandations roadmap V1, décision Go/No-Go pour scale-up

Lecture des jalons Chaque jalon est associé à une **décision Go/No-Go** : si le critère n'est pas franchi, le projet ne passe pas mécaniquement au jalon suivant. Cette discipline évite l'effet "tunnel" où l'on continue à développer sur des fondations défaillantes.

Pour ton oral : si on te demande « *et si J1 échoue, vous faites quoi?* », ta réponse est : « *Je révisé le backlog en conséquence avant d'attaquer le développement. Un MVP qui ignore les apprentissages Discovery est un MVP qui échoue à l'arrivée.* »

2.3 Échéancier (Gantt simplifié sur 12 semaines)

Le projet se déroule sur **12 semaines effectives**, organisées en **6 sprints de 2 semaines**. Le tableau ci-dessous présente la répartition des principaux chantiers.

Vue d'ensemble

Semaine	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
Sprint	Sp1	Sp1	Sp2	Sp2	Sp3	Sp3	Sp4	Sp4	Sp5	Sp5	Sp6	Sp6
Jalon		J1		J2		J3		J4		J5		J6

Chantiers par sprint

Sprint	Semaines	Thème principal	Chantiers clés
Sp1	S1-S2	Discovery & cadrage	Interviews utilisateurs, validation personas, raffinement backlog, finalisation Terraform
Sp2	S3-S4	Infrastructure cloud	Provisionnement AWS (ECS, ECR, S3, DynamoDB, Secrets Manager), déploiement Qdrant Cloud, pipeline ingestion OpenAgenda → S3 → Qdrant, environnement staging
Sp3	S5-S6	Cœur fonctionnel	Mémoire conversationnelle (LangGraph checkpointing + DynamoDB), contexte géographique (API BAN + filtre Qdrant), intégration Tavily Search comme tool agentique
Sp4	S7-S8	Qualité & observabilité	Intégration Langfuse, dashboards Snowflake (ingestion logs conversationnels), évaluation étendue (50 questions), tests de charge

Sprint	Semaines	Thème principal	Chantiers clés
Sp5	S9-S10	Préparation production	Hardening sécurité (RGPD, secrets, IAM least privilege), CI/CD GitHub Actions complète, déploiement production, monitoring CloudWatch, runbooks d'incident
Sp6	S11-S12	Post-lancement & bilan	Monitoring usage réel, ajustements rapides, rapport de bilan, recommandations roadmap V1, présentation aux parties prenantes

Représentation graphique (à générer en annexe)

Un Gantt visuel au format image sera produit via Genspark à partir du CSV en annexe (annexes/gantt_mvp.csv) et inséré ici dans la version finale du rapport.

2.4 Livrables par jalon

Chaque jalon est rattaché à des **livrables vérifiables**. Cette discipline garantit que “franchir un jalon” ne signifie pas simplement “avoir passé la date prévue”, mais bien “avoir produit les artefacts qui prouvent que l’étape est terminée”.

Jalon	Livrables associés
J0 — Kick-off	Rapport de cadrage validé (le présent document), repo GitHub initialisé, comptes cloud provisionnés, charte d'équipe, plan de communication parties prenantes

Jalon	Livrables associés
J1 — Discovery	Compte-rendu des interviews utilisateurs (synthèse qualitative), personas révisés v2, backlog raffiné (cf. section 3) avec user stories mises à jour, plan de tests utilisateurs MVP
J2 — Infra cloud	Code Terraform versionné, schéma d'architecture déployée (vs. cible), runbook d'opération (FAQ technique), environnement staging accessible avec URL et credentials sécurisés, premier pipeline d'ingestion exécuté avec succès
J3 — Cœur fonctionnel	Code applicatif des 4 fonctionnalités clés mergé, tests unitaires et d'intégration > 70% de couverture, démo enregistrée présentée en sprint review, documentation utilisateur v1
J4 — Monitoring & qualité	Dashboards Langfuse opérationnels, dashboards Snowflake avec 5 KPIs métier (cf. section 4), rapport d'évaluation étendu (50 questions, scores de pertinence, latence, coût), résultats des tests de charge
J5 — MVP en production	URL publique du MVP, certificat SSL valide, monitoring CloudWatch actif, premiers logs d'usage réel, registre des incidents post-lancement, communication de lancement validée par marketing
J6 — Bilan & roadmap	Rapport de bilan MVP avec métriques d'usage et de satisfaction, présentation aux parties prenantes, recommandations roadmap V1 prioritisées, décision Go/No-Go pour la suite documentée

Synthèse : la grille de pilotage

L'ensemble des éléments des sections 2.1 à 2.4 constitue la **grille de pilotage** que le Product Owner et le Tech Lead utiliseront en sprint review pour valider l'avancement. Cette grille est volontairement **visuelle et synthétique** pour permettre des décisions rapides en réunion de pilotage.

Note méthodologique : la combinaison "sprint (rythme) + jalon (étape)

+ livrable (preuve)” évite trois écueils classiques : - Le projet “qui avance bien” sans jamais rien livrer de tangible, - Le projet “qui livre” sans avoir validé d’étapes structurantes, - Le projet “qui franchit des étapes” mais sans rythme de production constant.

Annexe CSV pour le Gantt : annexes/gantt_mvp.csv

Section 3 — Macro backlog des fonctionnalités

Cette section présente l’ensemble des fonctionnalités identifiées pour le MVP, **priorisées selon la méthode MoSCoW** (Must-Have, Should-Have, Could-Have, Won’t-Have), **estimées en story points** (échelle de Fibonacci 1-2-3-5-8), et accompagnées de leurs **risques principaux et stratégies de mitigation**.

3.1 Méthode de priorisation MoSCoW

La priorisation MoSCoW est utilisée pour distinguer ce qui est **indispensable** au lancement du MVP de ce qui peut être différé sans compromettre la proposition de valeur.

Catégorie	Signification	Engagement
Must-Have	Indispensable au MVP, échec si absent	Livré dans le périmètre des 12 semaines, sans compromis
Should-Have	Important mais non bloquant	Livré si la capacité de l’équipe le permet, peut glisser au sprint suivant
Could-Have	Souhaitable, valeur secondaire	Livré uniquement si tous les Must et Should sont sécurisés
Won’t-Have (pour ce MVP)	Exclu volontairement de ce périmètre	Documenté dans la roadmap post-MVP

Note pédagogique : “Won’t-Have” ne signifie pas “abandonné”. Il signifie “pas dans ce MVP, mais explicitement reconnu et planifié pour une version ultérieure”. Cette discipline évite l’effet “scope creep” (élargissement non contrôlé du périmètre).

3.2 Échelle d’estimation en story points

SP	Complexité	Repère projet
1	Trivial	Modification de configuration, ajout de champ simple
2	Simple	Endpoint CRUD sur ressource existante
3	Modéré	Intégration d’API externe documentée

SP	Complexité	Repère projet
5	Complexe	Fonctionnalité métier nouvelle multi-composants
8	Très complexe	Intégration distribuée nouvelle avec persistance

Vélocité cible estimée : **~20 points par sprint** pour 2 ETP développeurs effectifs, à réajuster après les 2 premiers sprints sur la base de la vélocité observée.

3.3 Macro backlog — vue d'ensemble

ID	Epic	Fonctionnalité	MoSCoW	SP	Sprint cible	Risques principaux
F01	Données	Pipeline d'ingestion OpenAgenda → S3 (raw)	Must	3	Sp2	Quotas API source, format données changeant
F02	Données	Préprocessing + chunking documents	Must	3	Sp2	Qualité du chunking impacte la pertinence RAG
F03	Données	Vectorisation Mistral Embed → Qdrant Cloud	Must	5	Sp2	Coût embeddings, rate limit Mistral
F04	Données	Enrichissement géocodage via API BAN	Must	3	Sp2	Adresses manquantes ou ambiguës (~15% du jeu)
F05	Infra	IaC Terraform AWS (VPC, ECS, ECR, S3, DynamoDB, Secrets Manager)	Must	8	Sp2	Complexité Terraform, droits IAM à calibrer

ID	Epic	Fonctionnalité	MoSCoW	SP	Sprint cible	Risques principaux
F06	Infra	Pipeline CI/CD GitHub Actions (build, test, push ECR, deploy ECS)	Must	5	Sp2-Sp5	Premier déploiement complexe à debugger
F07	RAG	Recherche sémantique de base (LangGraph + Qdrant)	Must	3	Sp3	Repris du POC rag-events-2026
F08	RAG	Self-correction loop (grade documents + rewrite query)	Must	3	Sp3	Repris du POC rag-events-2026
F09	RAG	Filtrage géographique (rayon km) côté Qdrant	Must	5	Sp3	Précision géocodage en amont (cf. F04)
F10	RAG	Mémoire conversationnelle (LangGraph checkpointer + DynamoDB)	Must	8	Sp3	Cohérence sessions, RGPD historique conversationnel
F11	Agent	Tool Tavily Search (recherche web temps réel)	Must	5	Sp3	Coût API, qualité des sources, latence ajoutée

ID	Epic	Fonctionnalité	MoSCoW	SP	Sprint cible	Risques principaux
F12	Agent	Routing intelligent (RAG seul vs RAG + Web)	Must	5	Sp3	Faux positifs (web inutile), faux négatifs (web manquant)
F13	UX	Interface chat (Streamlit puis Next.js si temps)	Must	5	Sp3-Sp4	Choix techno UX, dépendance designer
F14	UX	Affichage sources et distance par résultat	Must	2	Sp4	Faible
F15	UX	Bouton feedback utilisateur ((+) / (-))	Must	2	Sp4	Faible
F16	Observabilité	Intégration Langfuse (traces + coûts + feedback)	Must	5	Sp4	Configuration auto-instrumentation LangGraph
F17	Observabilité	Pipeline logs → S3 → Snowflake	Must	5	Sp4	Volumétrie et coût Snowflake à calibrer
F18	Observabilité	Dashboard métier Snowflake (5 KPIs : volume, satisfaction, latence, coût, taux de réponse pertinente)	Must	5	Sp4	Qualité des données amont

ID	Epic	Fonctionnalité	MoSCoW	SP	Sprint cible	Risques principaux
F19	Qualité	Jeu d'évaluation étendu 50 questions	Must	3	Sp4	Subjectivité annotation, biais d'évaluateur
F20	Qualité	Tests de charge (Locust, montée à 100 utilisateurs concurrents)	Must	3	Sp4-Sp5	Coût LLM lors des tests, faux pic prod
F21	Sécurité	Conformité RGPD (consentement, anonymisation, TTL historique)	Must	5	Sp5	Validation DPO requise, impact sur fonctionnalités
F22	Sécurité	IAM least privilege + rotation secrets	Must	3	Sp5	Risque blocage si over-restriction
F23	Sécurité	Rate limiting + protection abus LLM (cap user)	Must	3	Sp5	Calibrage seuils
F24	Production	Déploiement public + nom de domaine + SSL	Must	3	Sp5	Propagation DNS, certificats
F25	Production	Runbooks d'incident (5 scénarios principaux)	Must	2	Sp5	Documentation à maintenir
F26	UX	Authentification utilisateur (AWS Cognito)	Should	5	Sp5	Complexité parcours, peut basculer post-MVP
F27	RAG	Multilingue FR/EN	Should	2	Sp4	Repris du POC

ID	Epic	Fonctionnalité	MoSCoW	SP	Sprint cible	Risques principaux
F28	UX	Streaming des réponses LLM (UX temps réel)	Should	3	Sp4	Compatibilité Streamlit limitée
F29	Observabilité	Alerting CloudWatch (latence, erreurs, coût)	Should	3	Sp5	Calibrage seuils, fatigue d'alertes
F30	RAG	Reranking via cross-encoder (qualité top-k)	Could	5	Sp5-Sp6	Latence ajoutée, coût
F31	UX	Suggestions de relance proactives	Could	3	Sp6	Qualité difficile à mesurer
F32	UX	Export résultats par email	Could	3	Sp6	Service SES à configurer
F33	Hors MVP	Notifications push événements	Won't	—	—	Roadmap V1
F34	Hors MVP	Réservation directe billetterie	Won't	—	—	Roadmap V1 (partenariats requis)
F35	Hors MVP	Application mobile native	Won't	—	—	Roadmap V2
F36	Hors MVP	Recommandations proactives par profil	Won't	—	—	Roadmap V1 (clustering utilisateurs)

3.4 Synthèse de la charge

Catégorie	Nombre d'items	Total story points	% du total
Must-Have	25	110	84 %
Should-Have	4	13	10 %
Could-Have	3	11	8 %
Won't-Have	4	—	—
Total estimé MVP	32	134 SP	100 %

À une vélocité estimée de ~20 SP par sprint sur 5 sprints de développement (Sp2 à Sp6), la capacité totale est de ~**100 SP**. La charge Must-Have (110 SP) est donc **légèrement supérieure** à la capacité, ce qui constitue un **risque planning à surveiller** dès le sprint 2. Les fonctionnalités Should et Could ne seront engagées qu'une fois la vélocité réelle observée.

Discipline projet : il est volontaire de planifier le Must-Have légèrement au-dessus de la capacité. Cela force l'équipe à reprioriser en continu plutôt qu'à se reposer sur une marge confortable qui inviterait à l'élargissement du périmètre.

3.5 Cartographie des risques principaux

Les risques identifiés ci-dessous sont issus de l'analyse des fonctionnalités et des dépendances externes du projet.

ID	Risque	Probabilité	Impact	Stratégie de mitigation
R1	Dépassement de coût LLM (embeddings + chat) en phase de chargement initial et tests de charge	Moyenne	Moyen	Cache d'embeddings, batch processing, plafond de coût mensuel sur Mistral + alerting Langfuse
R2	Quotas et rate limits sur APIs externes (Mistral, Tavily, BAN)	Élevée	Moyen	Retry avec backoff exponentiel, file d'attente côté ingestion, monitoring quotas

ID	Risque	Probabilité	Impact	Stratégie de mitigation
R3	Qualité du géocodage insuffisante (~15% adresses ambiguës)	Moyenne	Moyen	Fallback sur géocodage approximatif par ville, alerting sur taux d'échec
R4	Vélocité réelle inférieure à 20 SP/sprint	Moyenne	Élevé	Reprioritisation MoSCoW à chaque sprint review, descope Should/Could
R5	Non-conformité RGPD bloquante en validation DPO	Faible	Élevé	Validation DPO en J1 (Discovery) avant développement F10 (mémoire) et F21 (RGPD)
R6	Pertinence des réponses dégradée sous charge réelle	Moyenne	Élevé	Évaluation continue 50 questions + dashboard Langfuse + feedback utilisateur en boucle courte
R7	Coût Snowflake imprévu (volumétrie logs sous-estimée)	Faible	Moyen	Politique de rétention (90 jours), agrégation amont, monitoring quotidien des crédits
R8	Dépendance designer freelance (planning bloquant)	Moyenne	Moyen	Maquettes en J1, livraisons par lots, fallback Streamlit si Next.js indisponible

ID	Risque	Probabilité	Impact	Stratégie de mitigation
R9	Indisponibilité prolongée d'une dépendance critique (Mistral, Tavily)	Faible	Élevé	Documentation des alternatives (OpenAI, Anthropic pour LLM ; Exa, Brave pour search), abstraction du provider dans le code
R10	Dérive qualité du modèle dans le temps (drift)	Moyenne	Moyen	Évaluation hebdomadaire automatisée sur jeu de référence, alerte si baisse > 10%

Matrice de criticité

	Impact Faible	Impact Moyen	Impact Élevé
Probabilité Élevée	—	R2	—
Probabilité Moyenne	—	R1, R3, R7, R8, R10	R4, R6
Probabilité Faible	—	—	R5, R9

Les risques **R4** (vélocité insuffisante) et **R6** (dégradation qualité) sont les **deux risques majeurs** à surveiller en priorité car ils combinent probabilité moyenne et impact élevé. Ils justifient la mise en place très tôt des dashboards Langfuse et Snowflake (sprint 4, jalon J4).

3.6 Disposition Excel / CSV

Le backlog complet est disponible en annexe au format CSV importable dans Excel ou Google Sheets : annexes/backlog_mvp.csv. Ce fichier contient l'ensemble des champs (ID, Epic, Fonctionnalité, MoSCoW, Story Points, Sprint cible, Risque principal, Mitigation) et est mis à jour à chaque sprint review.

Fichier CSV à créer : annexes/backlog_mvp.csv

Section 4 — Architecture technique

Cette section présente l'architecture technique du MVP en quatre volets : la sélection du cloud provider, la sélection des composants applicatifs spécialisés, le schéma d'architecture déployée, et les stratégies transverses (déploiement, modularité, monitoring).

L'ensemble des choix techniques repose sur une **démarche de veille structurée** comparant les acteurs majeurs du marché selon des critères alignés sur les contraintes identifiées en section 1.

4.1 Choix du cloud provider

4.1.1 Panorama du marché en 2026 Le marché des fournisseurs cloud pertinents pour un projet data/IA français se structure autour de **deux catégories** :

Les hyperscalers mondiaux dominent le marché global avec 62% de part cumulée (DigitalOcean Cloud Market Report 2026) : - **AWS** : leader historique, croissance +18% en FY2025, écosystème le plus mature pour les workloads de production complexes. - **Microsoft Azure** : +25% YoY, intégration forte avec OpenAI (partenariat exclusif). - **Google Cloud Platform (GCP)** : +28% YoY, plus forte croissance, perçu comme leader sur les capacités IA/ML natives (Vertex AI, Gemini).

Les acteurs européens souverains prennent une part croissante sur le marché français, portés par les enjeux RGPD et de souveraineté numérique : - **Scaleway** (groupe Iliad/Free, France) : compute jusqu'à 4,8x moins cher qu'AWS selon le benchmark Callista de février 2026, offre GPU souveraine, datacenters certifiés SecNumCloud. - **OVHcloud** (France) : plus ancien acteur souverain, large catalogue, hausse tarifaire annoncée de 5-10% mi-2026.

4.1.2 Critères de comparaison Les critères de sélection ont été dérivés directement des contraintes identifiées en section 1.4 :

Critère	Justification	Pondération
Coût build + OPEX MVP	Budget contraint en alternance, MVP early-stage	Élevée
Free Tier / Free Trial	Validation à coût quasi-nul jusqu'au scale-up	Élevée
Écosystème services managés	Équipe restreinte, services managés > self-hosted	Élevée
Souveraineté et conformité RGPD	Données utilisateurs français	Moyenne

Critère	Justification	Pondération
Maturité IaC (Terraform)	Reproductibilité, démontre la maturité d'ingénierie	Moyenne
Compatibilité Snowflake	Snowflake retenu en data warehouse, doit s'intégrer simplement	Moyenne
Écosystème IA natif	Bonus si présent, non bloquant (LLM via Mistral externe)	Faible
Compétences disponibles	Mes compétences personnelles AWS, courbe d'apprentissage	Moyenne

4.1.3 Comparaison des options

Critère	AWS	GCP	Azure	Scaleway
Free Tier MVP	✓ 12 mois larges, déjà actif jusqu'à juillet 2026	△ \$300 sur 90 jours	△ \$200 sur 30 jours	△ Crédits ponctuels
Coût compute (vs référence AWS)	1× (référence)	0,9× à 1,1×	0,95× à 1,1×	~0,2× (4,8× moins cher)
Services managés conteneurs	ECS Fargate (standard de fait)	Cloud Run (excellent)	Container Apps	Serverless Containers
Base NoSQL serverless	DynamoDB (mature)	Firestore	Cosmos DB	Aucune équivalente managée
Stockage objet	S3	GCS	Blob Storage	Object Storage S3-compatible
Secrets managés	Secrets Manager + KMS	Secret Manager	Key Vault	Secret Manager
Intégration Snowflake	Excellente (natif via Snowpipe + connecteurs S3)	Très bonne	Bonne	Limitée (via S3-compatible)

Critère	AWS	GCP	Azure	Scaleway
Souveraineté FR/EU	Région eu-west-3 (Paris), pas SecNumCloud	Région europe-west9 (Paris)	Région France Central	✓ Souverain natif, SecNumCloud
Maturité IA native	Bedrock (multi-LLM)	Vertex AI + Gemini (leader perçu)	Azure OpenAI	Limitée
Maturité Terraform	✓ Provider le plus mature	✓ Très mature	✓ Très mature	△ Mature mais moins de ressources
Communauté et documentation	✓ Très large	✓ Large	✓ Large	△ Plus restreinte
Compétences personnelles	✓ Maîtrise existante	△ Notions	△ Notions	△ Notions

4.1.4 Décision : AWS retenu pour le MVP AWS est retenu comme cloud provider du MVP, pour cinq raisons articulées :

1. **Free Tier déjà actif jusqu'à juillet 2026**, ce qui couvre l'intégralité de la durée du MVP sans facture d'infrastructure significative. Cet avantage est temporellement aligné avec le projet et ne se retrouve sur aucun autre fournisseur.
2. **Maturité de l'écosystème de services managés** (ECS Fargate, DynamoDB, S3, Secrets Manager, CloudWatch) qui permet à une équipe restreinte de déployer une architecture production-grade sans gérer d'infrastructure bas niveau.
3. **Intégration native avec Snowflake** via Snowpipe et les connecteurs S3, qui simplifie le pipeline de logs conversationnels vers le data warehouse analytique (cf. section 4.3).
4. **Compétences personnelles existantes** sur la plateforme, ce qui réduit le risque de courbe d'apprentissage en milieu de projet (R4 — vitesse, identifié en section 3.5).
5. **Maturité du provider Terraform AWS** (le plus mature du marché), qui sécurise la stratégie d'Infrastructure-as-Code et démontre une maturité d'ingénierie attendue par un jury de niveau 7.

4.1.5 Ce qui n'est pas retenu et pourquoi Documenter explicitement les choix écartés est une discipline de veille technique. Les trois options non retenues sont :

GCP est techniquement supérieur sur les capacités IA natives (Gemini, Vertex AI), mais cet avantage est **neutralisé** dans notre architecture par le choix de Mistral AI comme fournisseur LLM externe. Le Free Trial GCP (\$300 sur 90 jours) ne couvre pas la durée du projet, et l'absence de compétences personnelles ajouterait un risque planning.

Azure présente un avantage clair pour les organisations investies dans l'écosystème

Microsoft (Active Directory, M365), ce qui n'est pas le cas de Puls-Events. L'avantage Azure OpenAI est neutralisé par le choix Mistral. Le Free Trial très court (30 jours) exclut une utilisation MVP.

Scaleway est un choix séduisant sur le plan de la souveraineté et du coût (4,8x moins cher qu'AWS sur le compute), et constitue un **candidat sérieux pour une migration post-MVP** une fois le produit validé. Il a été écarté pour le MVP pour trois raisons : absence de base NoSQL managée équivalente à DynamoDB (impact direct sur la fonctionnalité F10 — mémoire conversationnelle), maturité moindre des modules Terraform, et intégration Snowflake moins directe. Une **clause de réversibilité** est néanmoins intégrée dans la stratégie de modularité (cf. section 4.4) pour permettre ce basculement futur si le contexte économique ou réglementaire l'impose.

4.1.6 Région AWS retenue : eu-west-3 (Paris) Au sein d'AWS, la **région eu-west-3 (Paris)** est retenue pour trois raisons :

1. **Localisation des données en France**, en cohérence avec les contraintes RGPD et l'attente implicite de souveraineté pour une plateforme française.
2. **Latence réseau optimale** pour les utilisateurs français (< 20ms vs 40-60ms si Irlande).
3. **Disponibilité de tous les services requis** : ECS Fargate, DynamoDB, S3, Secrets Manager, CloudWatch, ECR, Cognito sont tous disponibles dans cette région depuis 2017.

Sous-section 4.2 — Choix des composants applicatifs (Groupe 1 : cœur IA)

4.2 Choix des composants applicatifs

L'architecture du MVP repose sur six composants externes ou spécialisés, regroupés en deux familles : le **cœur IA** (base vectorielle, fournisseur LLM, framework agentique) traité dans la sous-section 4.2, et les **composants de support** (monitoring LLM, data warehouse, search-for-LLM) traités dans la sous-section 4.3.

Pour chaque composant, la veille suit la même structure : critères de sélection alignés sur les contraintes du projet, comparaison des options, décision motivée, options écartées.

Posture méthodologique : privilégier la maturité communautaire

Au-delà des critères techniques propres à chaque composant, un critère transverse est appliqué à toutes les sélections : **la maturité de la communauté**. Cette posture est volontaire et repose sur trois bénéfices concrets pour un MVP en équipe restreinte :

- **Vélocité de développement** : un écosystème dominant signifie une documentation abondante, des exemples de production, des intégrations natives avec les outils tiers. Ce facteur mitige directement le risque R4 (vélocité insuffisante).
- **Pérennité du projet** : un outil porté par une entreprise solide ou une fondation, avec une croissance d'adoption confirmée, réduit le risque d'abandon technologique.

- **Disponibilité des compétences sur le marché** : facilite les recrutements futurs et la transmission entre développeurs.

Ce critère ne se substitue pas aux critères techniques mais agit comme **arbitre en cas de proximité technique entre options**.

4.2.1 Base de données vectorielle

Critères de sélection

Critère	Justification	Pondération
Performance recherche hybride (vecteurs + métadonnées)	F09 — filtrage géographique en un seul appel	Élevée
Service managé Cloud disponible	Équipe restreinte, pas de self-hosting K8s	Élevée
Free Tier durable pour MVP	Budget contraint	Élevée
Maturité communautaire (stars, adoption, doc)	Réduction risque R4	Élevée
Compatibilité native LangGraph / LangChain	Intégration sans code custom	Moyenne
Reprise sans friction depuis le POC rag-events-2026	Capitaliser sur l'existant	Moyenne
Performance pure (latence p95)	Critique pour UX < 5s	Moyenne

Comparaison

Critère	Qdrant	Pinecone	Weaviate	pgvector
Filtrage hybride en une requête	✓ Excellent	✓ Bon	✓ Excellent	△ Possible mais lourd
Free Tier Cloud	✓ 1 GB cluster gratuit illimité	△ Indexes limités	△ Sandbox 14 jours	✓ Gratuit (auto-hébergé sur RDS)
Maturité communautaire	✓ ~22k stars, croissance forte 2024-2026	✓ Standard historique mais closed-source	✓ ~11k stars	✓ Massif (Postgres)
Open-source	✓ Apache 2.0	✗ Fermé propriétaire	✓ BSD-3	✓ PostgreSQL

Critère	Qdrant	Pinecone	Weaviate	pgvector
Performance (Rust, benchmarks 2026)	✓ Top 3 rapidité	✓ Très bon	△ Bon	△ Limité à grande échelle
Intégration LangGraph	✓ Native	✓ Native	✓ Native	✓ Native
Self-hosting possible (réversibilité)	✓ Docker simple	✗	✓ Docker / K8s	✓ Postgres standard
Déjà utilisé dans POC rag-events-2026	✓	✗	✗	✗

Décision : Qdrant Cloud Qdrant Cloud est retenu comme base de données vectorielle du MVP, pour cinq raisons :

1. **Continuité avec le POC rag-events-2026** déjà validé sur 5 180 vecteurs avec 0% d'erreur sur 30 questions. La courbe d'apprentissage est nulle, le code est réutilisable directement.
2. **Free Tier généreux et durable** (1 GB de cluster managé sans limite de durée), suffisant pour 50 000 à 100 000 événements vectorisés en dimension 1024.
3. **Filtrage hybride performant** crucial pour la fonctionnalité F09 (filtrage géographique sur lat/lng + recherche sémantique en un seul appel API).
4. **Implémentation Rust** garantissant une latence basse même sous charge, élément clef pour l'UX < 5s.
5. **Open-source Apache 2.0 + self-hosting trivial** : en cas de nécessité de bascule (coût, souveraineté renforcée), un déploiement sur ECS ou sur Scaleway reste possible sans migration de format de données.

Ce qui n'est pas retenu Pinecone est l'acteur historique du marché, intégré nativement partout, mais est **closed-source** (lock-in fournisseur) et son free tier est plus limité dans le temps. Pour un MVP qui cherche à préserver l'optionalité, Qdrant offre un meilleur ratio qualité/prix/réversibilité.

Weaviate propose un excellent GraphQL et un module de hybrid search natif (BM25 + vectoriel), mais son free tier est limité à 14 jours, ce qui le disqualifie pour un MVP early-stage.

pgvector sur Postgres RDS est séduisant pour la simplicité (une seule base relationnelle + vectorielle), mais ses performances sont en retrait au-delà de quelques dizaines de milliers de vecteurs, et son ergonomie de filtrage hybride reste plus lourde que celle de Qdrant. Il sera **réévalué post-MVP** si l'on souhaite consolider les bases de données.

4.2.2 Fournisseur LLM (embeddings + génération)

Critères de sélection

Critère	Justification	Pondération
Qualité de génération en français	Marché cible français	Élevée
Hébergement / souveraineté européens	RGPD, attente implicite Puls-Events	Élevée
Coût par million de tokens	OPEX MVP maîtrisé	Élevée
Free Tier ou tarif d'entrée bas	Budget contraint	Moyenne
Maturité API (stabilité, SDK Python)	Vélocité dev	Moyenne
Modèles d'embeddings inclus	Éviter de multiplier les fournisseurs	Moyenne
Compatibilité OpenAI SDK	Permet une bascule rapide vers un autre fournisseur	Moyenne
Multilingue FR/EN	F27 — multilingue Should-Have	Faible

Comparaison

Critère	Mistral AI	OpenAI	Anthropic Claude	Gemini
Origine / hébergement	FR France + UE	US USA	US USA	US USA (régions UE possibles)
Qualité génération FR	✓ Native, excellente	✓ Excellente	✓ Excellente	✓ Excellente
Modèle d'embeddings inclus	✓ mistral-embed (1024-dim)	✓ text-embedding-3-*	✗ Pas de modèle embedding	✓ gemini-embedding
Coût LLM "small" (€/M tokens input)	~€0,20	\$0,15-\$2,50 selon modèle	\$0,80-\$3	\$0,10-\$0,30
Free Tier API	✓ Tier gratuit avec rate limit	△ Crédits ponctuels	△ Crédits ponctuels	✓ Tier gratuit
Conformité RGPD (DPA français)	✓ Disponible	✓ Disponible	✓ Disponible	✓ Disponible

Critère	Mistral AI	OpenAI	Anthropic Claude	Gemini
Streaming réponses	✓	✓	✓	✓
Compatibilité OpenAI SDK	✓ Endpoint compatible	✓ Natif	△ Via wrapper	△ Via wrapper
Maturité communautaire	✓ Forte croissance 2024-2026	✓ Standard de fait	✓ Standard en codage	✓ Forte adoption Google Cloud
Déjà utilisé dans POC	✓	✗	✗	✗

Décision : Mistral AI Mistral AI est retenu comme fournisseur LLM unique du MVP (embeddings + génération), pour cinq raisons :

1. **Continuité avec le POC** validé en production sur 5 000 événements, avec un score recall de 95% et 0% d'erreur sur 30 questions.
2. **Souveraineté française** alignée avec les attentes implicites d'une plateforme française et facilitant la gestion RGPD (DPA en français, données hébergées en UE).
3. **Modèle d'embeddings inclus** dans le même catalogue, ce qui évite de multiplier les fournisseurs et simplifie la gestion des clés API et de la facturation.
4. **Tier gratuit** suffisant pour la phase de vectorisation initiale et l'essentiel du trafic MVP early-stage.
5. **Compatibilité OpenAI SDK** : l'endpoint Mistral est compatible avec le SDK OpenAI, ce qui permet une **bascule rapide** vers OpenAI, Anthropic ou un autre fournisseur en modifiant uniquement la configuration (URL + clé). Cette propriété est cruciale pour mitiger le risque R9 (indisponibilité prolongée d'une dépendance critique).

Ce qui n'est pas retenu OpenAI (GPT-4.1, GPT-5) offre une qualité légèrement supérieure sur certains benchmarks complexes, mais le coût est 2 à 10× plus élevé selon les modèles, sans avantage déterminant pour les requêtes courtes typiques d'un chatbot d'événements (intent + génération de 200-500 tokens).

Anthropic Claude est excellent sur les tâches de raisonnement long et le code, mais (a) ne propose pas de modèle d'embeddings maison, ce qui obligerait à maintenir deux fournisseurs, et (b) son coût est positionné en premium (2-3× OpenAI).

Google Gemini présente le meilleur rapport coût/performance brut, mais son intégration cloud est optimisée pour GCP. Il constitue toutefois un **plan B sérieux** documenté dans la stratégie de réversibilité (cf. section 4.4).

4.2.3 Framework d'orchestration agentique

Critères de sélection

Critère	Justification	Pondération
Support des workflows stateful (mémoire conversationnelle)	F10 — mémoire conversationnelle Must-Have	Élevée
Maturité communautaire et adoption	Réduction risque R4	Élevée
Mécanismes de self-correction et de routing conditionnel	F08, F12 — déjà dans le POC	Élevée
Persistance des états (checkpointing) avec backends pluggables	DynamoDB pour mémoire conversationnelle	Élevée
Sécurité (pas d'exécution de code arbitraire par l'agent)	Production sensible	Élevée
Intégration native avec les outils d'observabilité	Langfuse — F16	Moyenne
Continuité avec le POC rag-events-2026	Capitaliser sur l'existant	Moyenne

Comparaison

Critère	LangGraph	LlamaIndex Workflows	CrewAI	smolagents (HF)
Paradigme	Graphe d'états	Workflows event-driven	Multi-agents collaboratifs par rôle	Code-acting agents
Workflows stateful	✓ Natif via state machine	✓ Via WorkflowContext	⚠ Limité	⚠ Limité
Self-correction conditionnelle	✓ Excellent (nœuds + conditions)	✓ Bon	⚠ Possible	⚠ Implicite via re-prompt
Checkpointing pluggable	✓ Memory, SQLite, Postgres, DynamoDB	⚠ Limité aux backends fournis	⚠ Limité	✗
Maturité communautaire	✓ Standard de fait 2025-2026	✓ Forte (data-centric)	✓ Croissance forte	⚠ Récent (fin 2024)
Sécurité production	✓ Pas d'exécution code	✓ Pas d'exécution code	✓ Pas d'exécution code	⚠ Exécute du code Python généré par le LLM

Critère	LangGraph	LlamaIndex Workflows	CrewAI	smolagents (HF)
Intégration Langfuse	✓ Native auto-instrumentation	✓ Disponible	✓ Disponible	△ Manuelle
Suggestion explicite de Jérémy	✗	✗	✗	✓
Déjà utilisé dans POC rag-events- 2026	✓	✗	✗	✗

Décision : LangGraph LangGraph est retenu comme framework d'orchestration agentique, pour quatre raisons articulées :

1. **Continuité avec le POC** : le pipeline self-correction (analyze → retrieve → grade → rewrite → generate) est déjà implémenté et évalué. Le passer à un autre framework représenterait une réécriture sans valeur ajoutée fonctionnelle.
2. **Checkpointing DynamoDB natif** : LangGraph propose nativement un système de persistance d'état avec des backends pluggables, ce qui est précisément ce qui nous permet d'implémenter la fonctionnalité F10 (mémoire conversationnelle) sans code custom.
3. **Sécurité production** : LangGraph n'exécute pas de code généré par le LLM, contrairement à smolagents. Pour un MVP en production accessible publiquement, c'est un critère de sécurité non négociable.
4. **Maturité communautaire** : LangGraph est devenu en 2025-2026 le framework dominant pour les workflows stateful complexes, avec un écosystème d'intégrations (Langfuse, AWS, Snowflake) plus large que ses concurrents.

Le cas particulier de smolagents (suggestion de Jérémy) La suggestion explicite de Jérémy dans le mail initial — *“intégrer des capacités de recherche avancées, pourquoi ne pas explorer la librairie smolagent de Hugging Face?”* — appelle une réponse documentée.

smolagents est une librairie élégante publiée par Hugging Face fin 2024, dont la spécificité est de faire écrire et exécuter du **code Python** par le LLM (paradigme “code-acting agents”). Cette approche est techniquement séduisante pour des tâches de raisonnement complexe, mais présente **trois inconvénients bloquants** dans notre contexte MVP :

- **Sécurité** : l'exécution de code Python généré par un LLM exposé à des entrées utilisateurs publiques constitue une surface d'attaque significative (injection, exfiltration, déni de service). Une sandboxing rigoureuse serait nécessaire, augmentant la complexité.
- **Sur-puissance pour le cas d'usage** : nos workflows (RAG + routing + recherche web) sont déterministes par nature et bien modélisés par un graphe d'états. L'écriture de code arbitraire par l'agent n'apporte rien.

- **Discontinuité avec le POC** : adopter smolagents impliquerait une réécriture complète du pipeline, sans bénéfice fonctionnel.

En revanche, **smolagents reste un candidat intéressant pour des fonctionnalités post-MVP** plus exploratoires (par exemple, un agent qui agrège des sources hétérogènes avec analyse statistique ad-hoc des événements). Cette ouverture est documentée dans la roadmap.

Ce qui n'est pas retenu LlamaIndex Workflows est techniquement très proche de LangGraph mais plus orienté “data ingestion” que “agentic workflow”. Notre besoin penche clairement vers l’agentique.

CrewAI est excellent pour les architectures multi-agents avec rôles distincts (chercheur + rédacteur + critique...), ce qui n’est pas notre cas d’usage. Notre architecture est mono-agent avec routing conditionnel, pour lequel LangGraph est plus adapté.

4.3 — Choix des composants applicatifs (Groupe 2 : composants de support)

Le groupe 2 couvre les trois composants qui entourent le cœur IA :

- l’observabilité LLM,
- le data warehouse analytique,
- et la recherche web temps réel.

Ces composants ne participent pas directement à la génération de la réponse mais sont indispensables à la **production-readiness** du MVP : sans monitoring on ne pilote pas, sans data warehouse on n’analyse pas, sans recherche web on ne répond pas aux questions hors corpus.

4.3.1 Monitoring LLM (LLM Observability)

Le besoin Un système RAG en production doit être observable sur trois niveaux spécifiques aux LLM, qui ne sont pas couverts par les outils classiques d’APM (CloudWatch, Datadog) :

- **Traces** : reconstitution complète du parcours d’une requête utilisateur dans le pipeline (analyze → retrieve → grade → rewrite → generate), avec les prompts, contextes, tokens et latences à chaque étape.
- **Qualité** : feedback utilisateur (F15 — boutons (+)/(-)), scores d’évaluation automatique (RAGAS), détection de dégradation qualité dans le temps (R10 — drift modèle).
- **Coûts** : suivi du coût LLM par utilisateur, par fonctionnalité, par session, avec capacité d’alerting en cas d’anomalie (R1 — dépassement coût LLM).

Critères de sélection

Critère	Justification	Pondération
Open-source et self-hostable	Maîtrise des coûts, contrôle des données conversationnelles (RGPD)	Élevée
Intégration native LangGraph	F16 — instrumentation sans code custom	Élevée
Free Tier Cloud utilisable	Démarrage rapide en MVP avant self-hosting	Élevée
Couverture des 3 niveaux (traces + qualité + coûts)	Approche unifiée	Élevée
Capture du feedback utilisateur ((+)/(-))	F15 — collecte feedback	Élevée
Maturité communautaire	Pérennité et documentation	Moyenne
Capacité d'évaluation (RAGAS, custom)	F19 — évaluation étendue 50 questions	Moyenne
Dashboards prêts à l'emploi	Réduction du temps de mise en place	Moyenne

Comparaison

Critère	Langfuse	Arize Phoenix	LangSmith
Modèle	Open-source MIT + Cloud	Open-source Elastic License + Arize AX SaaS	SaaS propriétaire LangChain Inc.
Self-hosting gratuit	✓ Total (Docker / K8s)	✓ Phoenix gratuit	✗ Plan gratuit limité, payant au-delà
Free Tier Cloud	✓ 50k observations/mois	△ Limité	△ Plan Developer payant depuis 2024
Intégration LangGraph	✓ Native auto-instrumentation	✓ Native via OpenTelemetry	✓ Native (même éditeur)
Traces complètes pipeline	✓	✓	✓
Capture feedback utilisateur	✓ SDK simple	△ Possible mais moins central	✓
Suivi coûts LLM par utilisateur	✓	✓	✓

Critère	Langfuse	Arize Phoenix	LangSmith
Évaluation automatique (RAGAS)	✓ Module Evals natif	✓ Module Evals natif	✓
Maturité communautaire	✓ ~9k stars, croissance forte 2024-2026	✓ Adossé à Arize AI	✓ Standard chez les utilisateurs LangChain
Dépendance fournisseur tiers	△ Possible (Cloud) ou ✗ (self-host)	△ Possible (AX) ou ✗ (Phoenix)	△ Forte (closed-source)
Conformité RGPD (self-host UE)	✓ Possible	✓ Possible	✗ Données chez LangSmith

Décision : Langfuse Langfuse est retenu comme plateforme d'observabilité LLM, pour quatre raisons :

1. **Open-source MIT entièrement self-hostable**, ce qui garantit
 - (a) un coût marginal nul à terme, (b) le stockage des traces conversationnelles sous notre propre contrôle (impératif RGPD pour des conversations utilisateurs), et (c) l'absence de lock-in fournisseur.
2. **Free Tier Cloud généreux** (50 000 observations par mois) permettant de démarrer immédiatement sans déployer l'infrastructure self-hosted, puis de basculer vers un déploiement self-hosté quand le volume le justifie.
3. **Intégration LangGraph native avec auto-instrumentation** : une ligne de configuration suffit pour capturer toutes les traces du pipeline, sans modification du code applicatif. Cela accélère F16 et réduit le risque R4 (vélocité).
4. **Couverture native du cycle complet** : traces, feedback utilisateur via SDK, suivi coût par modèle, évaluation automatique avec module RAGAS intégré. Pas besoin de multiplier les outils.

Stratégie d'usage retenue Une stratégie en **deux temps** est adoptée pour optimiser le rapport vélocité/coût :

- **Sprint 4 (J4)** : intégration de Langfuse Cloud (Free Tier) pour démarrer immédiatement la collecte de traces et de feedback utilisateur sans délai infrastructure.
- **Post-MVP** : bascule vers Langfuse self-hosté sur ECS Fargate si le volume dépasse 50k observations/mois ou si la conformité RGPD impose une localisation interne stricte. Cette bascule est planifiée comme une simple substitution de variable d'environnement (URL d'endpoint).

Ce qui n'est pas retenu **Arize Phoenix** est techniquement excellent et basé sur OpenTelemetry (standard d'observabilité ouvert), ce qui constitue un atout architectural. Il a été écarté pour deux raisons : (a) son intégration LangGraph est moins fluide que celle de Langfuse (passage par OpenTelemetry), et (b) sa communauté est plus orientée "data science / ML traditional" que "LLM apps".

LangSmith est l'outil natif de l'écosystème LangChain et offre l'intégration la plus simple, mais sa nature **SaaS propriétaire avec passage en plan payant en 2024** crée un risque coût et un lock-in fournisseur en contradiction avec notre stratégie de réversibilité.

4.3.2 Data Warehouse analytique

Le besoin Le data warehouse répond au persona P4 (équipe produit interne) identifié en section 1.6 : permettre l'analyse exploratoire des logs conversationnels, la construction de dashboards métier (F18 — 5 KPIs), et l'identification des requêtes mal traitées pour piloter l'amélioration continue.

Distinction importante : le data warehouse n'est pas la base opérationnelle du chatbot (qui est DynamoDB pour la mémoire et Qdrant pour les vecteurs). Il est la base **analytique** alimentée en aval par les logs produits par l'application et exportés depuis Langfuse et CloudWatch.

Critères de sélection

Critère	Justification	Pondération
Free Trial / Free Tier permettant un MVP gratuit	Budget constraint	Élevée
Intégration native avec AWS S3	Pipeline ingestion logs → S3 → DW	Élevée
Maturité communautaire et compétences en France	Recrutement, transmission, mutualisation	Élevée
Capacités SQL standard	Compétence Data Engineer transférable	Moyenne
Pricing par usage (consommation) plutôt que provisionné	Coût bas en MVP early-stage	Moyenne
Capacités IA natives (cf. Cortex AI sur Snowflake)	Bonus pour futurs use cases analytiques	Faible
Pas de dépendance forte à un cloud provider unique	Réversibilité	Moyenne

Comparaison

Critère	Snowflake	BigQuery (GCP)	Amazon Redshift
Free Trial / Free Tier	✓ \$400 ou 30 jours	△ 1 TB requêtes/mois mais nécessite GCP	✗ Pas de free tier durable
Modèle de pricing	Pay-per-use (crédits compute + storage séparés)	Pay-per-query	Provisionné (cluster) ou serverless
Intégration AWS S3	✓ Native via Snowpipe	△ Via Storage Transfer	✓ Native (même cloud)
Multi-cloud (AWS + GCP + Azure)	✓ Natif	✗ GCP-only	✗ AWS-only
Maturité communauté France	✓ Forte présence Paris, événements, Snowflake Summit Paris	✓ Présente	✓ Présente mais moins visible
Capacités IA natives	✓ Cortex AI (LLM, embeddings, vector search) en SQL	✓ Vertex AI integration, BigQuery ML	✓ Redshift ML (SageMaker)
Séparation compute/storage	✓ Native (élasticité)	✓ Native	△ Selon mode
Conformité RGPD UE	✓ Région Paris disponible	✓ Région Paris	✓ Région Paris
Compétences sur le marché FR	✓ Marché actif, forte demande	✓ Marché actif	△ Plus rare

Décision : Snowflake **Snowflake est retenu comme data warehouse analytique du MVP**, pour quatre raisons :

1. **Free Trial à \$400 de crédits** suffisant pour couvrir plusieurs mois d'usage MVP early-stage (volumétrie estimée en section 5).
2. **Indépendance vis-à-vis du cloud provider** : Snowflake fonctionne sur AWS, GCP et Azure de manière transparente. En cas de bascule cloud post-MVP (notamment vers Scaleway ou autre provider européen), le data warehouse reste identique. Cette propriété renforce la stratégie de réversibilité globale du projet.
3. **Communauté française active** : Snowflake dispose d'une présence forte à Paris avec un écosystème de partenaires, des événements communautaires (Snowflake Summit, meetups), et un marché actif de compétences. Cela facilite à la fois le recrutement et la montée en compétences personnelle.
4. **Modèle pay-per-use** parfaitement adapté à un MVP : on ne paye que ce qu'on consomme, sans cluster provisionné en permanence. Couplé à la séparation compute/storage, on peut stocker beaucoup pour très peu et ne consommer du compute que lors des analyses.

Stratégie d'usage retenue Le pipeline analytique est conçu pour être **non bloquant pour le chemin critique du chatbot** :

- Les logs applicatifs sont écrits en temps réel par l'application vers CloudWatch.
- Un batch quotidien exporte les logs vers S3 (format Parquet).
- Snowpipe ingère automatiquement les nouveaux fichiers S3 dans Snowflake.
- Les dashboards (F18) interrogent Snowflake en SQL standard.

Cette architecture découplée garantit que toute défaillance du data warehouse n'impacte pas l'expérience utilisateur.

Ce qui n'est pas retenu **BigQuery** est techniquement excellent et le Free Tier est plus généreux en absolu (1 TB de requêtes/mois). Il est écarté pour trois raisons : (a) il nécessite de provisionner et gérer un compte GCP en parallèle d'AWS, ajoutant de la complexité opérationnelle ; (b) l'intégration avec S3 (notre source) est moins directe ; (c) il enferme le projet dans l'écosystème GCP, contradiction avec notre stratégie de réversibilité.

Redshift est la solution native AWS et serait le choix le plus simple en termes d'intégration. Il est écarté pour deux raisons : (a) absence de free tier durable, donc coût immédiat non négligeable ; (b) lock-in AWS qui ne nous laisse pas l'optionnalité de bascule cloud post-MVP.

Note pour ton oral : si on te demande « *pourquoi pas Redshift puisque vous êtes sur AWS ?* », ta réponse est : « *Précisément parce que je suis sur AWS pour l'opérationnel, je préfère découpler l'analytique sur un acteur cloud-agnostic. Cela me donne la liberté future de migrer l'opérationnel sans toucher à l'analytique, ou inversement. C'est une discipline d'architecture qui paye à long terme.* »

4.3.3 Recherche web temps réel (Search-for-LLM)

Le besoin La fonctionnalité F11 (Tool de recherche web) répond au cas d'usage UC4 (persona Sophie, voyageuse week-end) : permettre au chatbot de répondre sur des événements **non indexés dans notre corpus OpenAgenda**, soit parce qu'ils sont trop récents, soit parce qu'ils proviennent de sources alternatives (office du tourisme local, presse régionale, réseaux sociaux d'organismes).

Le routage vers la recherche web (F12) est décidé par l'agent LangGraph lorsque le grade des documents Qdrant est insuffisant après une éventuelle réécriture de requête.

Critères de sélection

Critère	Justification	Pondération
Free Tier API exploitable en MVP	Budget contraint	Élevée

Critère	Justification	Pondération
Qualité des résultats pour LLM (réponses structurées, contenu propre)	Pertinence des réponses chatbot	Élevée
Indépendance de l'index (pas Google/Bing only)	Diversité des sources, souveraineté indirecte	Moyenne
Performance sur benchmarks agentic search 2026	Validation par des tiers	Moyenne
Stabilité du fournisseur (pas de rachat / pivot récent)	R9 — indisponibilité prolongée	Élevée
Maturité communautaire	Pérennité	Moyenne
Latence p95	UX < 5s	Moyenne
Couverture FR / contenu francophone	Marché cible	Élevée

Comparaison

Critère	Brave Search API	Tavily	Exa
Free Tier	✓ 2 000 requêtes/mois	✓ 1 000 requêtes/mois	△ Crédits ponctuels
Index	✓ Indépendant (Brave Index propre)	△ Aggrégateur de sources tierces	△ Aggrégateur orienté neural search
Réponses optimisées LLM	△ Bon, JSON structuré	✓ Excellent (réponses pré-résumées)	✓ Excellent (sémantique)
Benchmark agentic search 2026	✓ Leader (score 14.89)	△ Compétitif	△ Compétitif
Stabilité fournisseur	✓ Brave Software (entreprise stable, modèle économique propre)	△ Racheté par Nebius en 2025, alternatives recherchées par la communauté	✓ Stable (startup financée)
Souveraineté / éthique	✓ Index indépendant, respect vie privée affiché	△ Dépendant de sources tierces	△ Dépendant de sources tierces

Critère	Brave Search API	Tavily	Exa
Couverture FR	✓ Bonne (index global)	✓ Bonne	✓ Bonne
Maturité communautaire	✓ Forte	✓ Forte (mais incertitude post-acquisition)	✓ Croissance

Décision : Brave Search API **Brave Search API est retenu comme fournisseur de recherche web temps réel**, pour quatre raisons :

1. **Free Tier le plus généreux du panel** (2 000 requêtes/mois) et tarifs payants compétitifs au-delà, ce qui sécurise le budget MVP et le post-MVP early-stage.
2. **Leader du benchmark agentic search 2026** (aimultiple, score 14.89), validant la qualité technique par une source tierce indépendante.
3. **Index indépendant** (Brave maintient son propre index web, distinct de Google et Bing), ce qui apporte (a) une diversité de sources, (b) une indépendance vis-à-vis des géants du search, et (c) une cohérence avec l'argument de souveraineté du projet.
4. **Stabilité du fournisseur** : Brave Software a un modèle économique propre (navigateur, BAT, ads sans tracking) qui ne dépend pas du chiffre d'affaires de l'API. À l'inverse, Tavily a été **racheté par Nebius en 2025**, créant une incertitude stratégique que la communauté constate depuis (recherche active d'alternatives). Ce point est directement aligné avec le risque R9 (indisponibilité prolongée d'une dépendance critique).

Note sur le contexte communautaire L'intuition initiale concernant l'usage massif de Brave dans l'écosystème des agents (notamment certains exemples communautaires Anthropic ou intégrations MCP) est confirmée par la veille : Brave Search API s'est imposé en 2025-2026 comme **l'un des fournisseurs de search-for-LLM les plus populaires** chez les développeurs construisant des agents en production, en grande partie grâce à sa combinaison de free tier généreux, qualité de résultats, et positionnement éthique. Il est utilisé dans de nombreuses intégrations MCP (Model Context Protocol) et recommandé dans les guides d'agents 2026.

Le Claude Agent SDK, pour sa part, utilise par défaut l'outil web_search natif d'Anthropic, mais Brave reste l'alternative tierce la plus couramment recommandée et intégrée.

Stratégie d'usage retenue L'appel à Brave Search est conditionnel et déclenché par le nœud LangGraph route_to_web uniquement quand :

- Le score de pertinence du retrieval Qdrant est inférieur à un seuil défini (typiquement 0,5 sur cosine similarity).
- Et la première réécriture de requête n'a pas significativement amélioré ce score.

Cette logique garantit que le coût et la latence ajoutés par Brave ne sont engagés que lorsque le RAG seul est insuffisant, mitigant le risque R1 (coût) tout en améliorant la

couverture fonctionnelle.

Ce qui n’est pas retenu **Tavily** était initialement le standard de fait pour la recherche optimisée LLM, avec des réponses pré-résumées de qualité. Il est écarté principalement pour le risque stratégique post-acquisition Nebius 2025, qui a poussé une partie de la communauté à chercher des alternatives, ainsi que pour un free tier moins généreux que Brave.

Exa propose une approche intéressante de “neural search” (recherche sémantique sur l’index web), ce qui est techniquement séduisant pour les agents. Il est écarté principalement pour l’absence de free tier durable et pour un index plus restreint sur le contenu francophone.

4.4 — Schéma d’architecture (description textuelle) schéma en quatre vues

Cette sous-section présente l’architecture cible du MVP sous quatre vues complémentaires, en cohérence avec l’approche C4 de modélisation d’architecture logicielle. Chaque vue répond à une question spécifique et s’adresse à une audience identifiée.

Vue	Question	Audience
V1	Quelle est l’architecture globale en couches ?	Tech Lead, Architecte
V2	Comment les données d’événements arrivent-elles dans la base vectorielle ?	Data Engineer
V3	Que se passe-t-il quand un utilisateur pose une question ?	Dev + Product Owner
V4	Comment les logs sont-ils exploités pour le pilotage ?	Data Analyst + Product

Pourquoi quatre vues plutôt qu’un seul schéma : un schéma unique cherchant à tout représenter devient illisible. La séparation en vues complémentaires (inspirée du modèle C4 de Simon Brown) permet à chaque lecteur de comprendre ce qui le concerne sans surcharge cognitive.

4.4.1 Vue V1 — Vue d’ensemble en couches

L’architecture s’organise en **six couches logiques**, du plus proche de l’utilisateur au plus profond de l’infrastructure :

Couche 1 — Accès utilisateur (Edge) - Amazon Route 53 : DNS public, gestion du domaine chat.puls-events.fr. - **AWS Certificate Manager** : émission et renouvellement automatique du certificat SSL/TLS. - **Application Load Balancer (ALB)** : point d'entrée HTTPS, terminaison SSL, routage vers les conteneurs ECS, gestion des healthchecks.

Couche 2 — Application (Compute) - Amazon ECS Fargate : exécution conteneurisée de l'application chatbot (image Docker stockée dans ECR), serverless (pas de gestion des EC2), avec auto-scaling horizontal selon la charge CPU/mémoire. - **Amazon ECR (Elastic Container Registry)** : registre privé des images Docker buildées par la CI/CD. - Application interne : code Python orchestré par **LangGraph**, exposant une API HTTP via FastAPI (backend) et servant l'interface **Streamlit** (frontend MVP).

Couche 3 — Données opérationnelles (Run-time state) - Qdrant Cloud : base de données vectorielle hébergée (Free Tier 1 GB), accessible via HTTPS depuis ECS, contenant les ~5 000 événements vectorisés (1024 dimensions) avec métadonnées (ville, région, lat/lng, date, catégorie). - **Amazon DynamoDB** : base NoSQL serverless, table conversations contenant les états LangGraph (checkpointner) avec TTL automatique 30 jours pour conformité RGPD. Clé primaire session_id, clé de tri timestamp. - **Amazon S3** : trois buckets distincts pour la séparation des données — puls-events-raw (données OpenAgenda brutes), puls-events-processed (chunks prétraités), puls-events-logs (logs applicatifs exportés vers Snowflake).

Couche 4 — Intelligence IA (externe) - Mistral AI API : embeddings (mistral-embed 1024-dim) et génération (mistral-small-latest), accédée via HTTPS depuis ECS, clés stockées dans Secrets Manager. - **Brave Search API** : recherche web temps réel, appelée conditionnellement par le nœud route_to_web de LangGraph uniquement lorsque le retrieval Qdrant est insuffisant. - **API BAN (gouv.fr)** : géocodage des adresses d'événements en phase d'ingestion (lat/lng), gratuit et souverain.

Couche 5 — Sécurité et configuration - AWS Secrets Manager : stockage chiffré des clés API (Mistral, Brave, Qdrant, Snowflake), rotation programmée, accès restreint par IAM. - **AWS IAM** : rôles à privilèges minimaux (least privilege) pour chaque composant — rôle d'exécution ECS, rôle de tâche, rôle d'ingestion, rôle d'export logs. - **AWS VPC** : réseau privé virtuel avec sous-réseaux publics (ALB) et privés (ECS), security groups pour cloisonnement.

Couche 6 — Observabilité et analytique - Langfuse Cloud : observabilité LLM (traces, coûts, feedback) en SaaS Free Tier, basculable vers self-hosted post-MVP. - **Amazon CloudWatch** : logs applicatifs, métriques infrastructure (CPU, mémoire, latence ALB), alarmes (latence > 5s, taux d'erreur > 2%). - **Snowflake** : data warehouse analytique, alimenté par Snowpipe depuis le bucket S3 puls-events-logs. Persona P4 (équipe produit) y construit ses dashboards (F18).

Transversal — Infrastructure as Code et CI/CD - Terraform : code IaC versionné dans le repo, provisionnement complet de l'infrastructure AWS de manière reproductible. - **GitHub Actions** : pipeline CI/CD — build Docker → push ECR → deploy ECS via aws ecs update-service → tests de smoke en staging → promotion en production manuelle pour le MVP.

4.4.2 Vue V2 — Flux d'ingestion des données

Ce flux est exécuté **une première fois lors du déploiement initial** (bootstrap), puis **périodiquement** (CRON quotidien) pour rafraîchir les événements.

Étape 1 — Extraction - Source : API OpenDataSoft (agrégateur OpenAgenda), pas de clé requise. - Composant exécutant : tâche ECS planifiée (EventBridge trigger) ou Lambda selon le volume. - Sortie : fichiers JSON bruts déposés dans `s3://puls-events-raw/events_YYYYMMDD.json`.

Étape 2 — Préprocessing et enrichissement géographique - Lecture des fichiers bruts depuis S3. - Nettoyage (déduplication, validation des champs obligatoires, filtrage des événements expirés). - Chunking textuel (chunks de ~915 caractères, overlap 150 caractères, reprise du paramétrage validé dans rag-events-2026). - Enrichissement par appel à l'**API BAN** pour géocoder les adresses textuelles en coordonnées (lat/Ing). - Sortie : fichiers Parquet structurés dans `s3://puls-events-processed/chunks_YYYYMMDD.parquet`.

Étape 3 — Vectorisation et indexation - Lecture des chunks prétraités. - Appel batch à **Mistral Embed API** pour produire les vecteurs 1024-dim. - Upsert dans **Qdrant Cloud** avec métadonnées complètes (ville, région, lat/Ing, date, catégorie, source URL). - Création d'un index payload sur les champs filtrables (région, ville, date, lat/Ing) pour les requêtes hybrides.

Étape 4 — Validation et notification - Tests automatisés post-ingestion (cf. F19) : volume attendu, taux de géocodage réussi (> 85%), distribution géographique. - Notification vers SNS en cas d'anomalie (alerte équipe data).

4.4.3 Vue V3 — Flux d'une conversation utilisateur

C'est le flux **temps réel** au cœur du chatbot. Il illustre l'orchestration LangGraph et l'usage conditionnel des composants externes.

Étape 1 — Entrée utilisateur - L'utilisateur saisit une question dans l'interface Streamlit (par exemple : *"Une expo gratuite à Lyon ce week-end?"*). - La requête atteint l'**ALB**, puis le conteneur **ECS Fargate** exécutant l'application. - Récupération du `session_id` (cookie ou query param), géolocalisation utilisateur si disponible (consentement préalable).

Étape 2 — Chargement de la mémoire conversationnelle - L'application interroge **DynamoDB** via le checkpoint LangGraph pour récupérer l'état précédent de la session (historique des derniers échanges, préférences déduites). - Si nouvelle session, état vide initialisé.

Étape 3 — Analyse de la requête (nœud analyze_query) - Appel à **Mistral Small** pour extraire l'intent, la ville, la région, la catégorie, la temporalité. - Si une localisation est mentionnée, appel optionnel à l'**API BAN** pour géocoder.

Étape 4 — Retrieval hybride (nœud retrieve) - Génération du vecteur de la requête via **Mistral Embed**. - Requête à **Qdrant Cloud** combinant : recherche

par similarité cosinus + filtre métadonnées (région/ville/date) + optionnellement filtre géographique par rayon (lat/long + haversine). - Récupération des top-K documents (K=8 par défaut).

Étape 5 — Évaluation de pertinence (nœud grade_documents) - Appel à **Mistral Small** pour juger si les documents retrouvés sont suffisamment pertinents.

- **Branchement conditionnel** : - Si pertinents → étape 7 (génération). - Si non pertinents et pas encore de réécriture → étape 6a (rewrite query). - Si non pertinents et déjà une réécriture → étape 6b (web search).

Étape 6a — Réécriture de requête (nœud rewrite_query) - Appel à **Mistral Small** pour reformuler la question (synonymes, élargissement de catégories). - Retour à l'étape 4 avec la nouvelle requête.

Étape 6b — Recherche web temps réel (nœud route_to_web) - Appel à **Brave Search API** avec la requête enrichie (ville + thématique + date). - Récupération des top-5 résultats web (titres, snippets, URLs). - Ces résultats sont injectés comme contexte additionnel pour l'étape 7.

Étape 7 — Génération de la réponse (nœud generate) - Construction du prompt final : système (rôle, contraintes anti-hallucination, date courante) + contexte (documents Qdrant et/ou résultats Brave) + historique conversationnel (DynamoDB) + question utilisateur. - Appel à **Mistral Small** avec streaming. - La réponse est renvoyée à l'utilisateur, avec affichage des sources et de la distance (F14).

Étape 8 — Sauvegarde de l'état et observabilité - L'état mis à jour (question, réponse, contexte utilisé) est persisté dans **DynamoDB** par le checkpoint LangGraph. - Une trace complète de l'exécution est envoyée à **Langfuse** (latence par nœud, tokens consommés, coût estimé, documents retrouvés). - L'utilisateur peut envoyer un feedback ((+)/(-), F15) qui est associé à la trace Langfuse.

Étape 9 — Log applicatif pour analytique - Un log structuré JSON est écrit dans **CloudWatch** (anonymisé : pas d'identifiant utilisateur direct, conformité RGPD). - Le log contiendra : timestamp, session_id hashé, intent détecté, ville/région ciblée, nombre de documents retrouvés, utilisation web search oui/non, latence, feedback éventuel.

4.4.4 Vue V4 — Flux analytique

Ce flux est **asynchrone et découplé** du chemin critique utilisateur, conformément à la stratégie présentée en 4.3.2.

Étape 1 — Collecte des logs - Les logs applicatifs structurés sont écrits dans **CloudWatch Logs** en temps réel.

Étape 2 — Export quotidien vers S3 - Un **CloudWatch Logs subscription filter** exporte les logs vers le bucket puls-events-logs au format JSON, partitionné par date. - Une étape de conversion en **Parquet** (via Lambda ou tâche ECS) optimise le stockage et la requêtabilité.

Étape 3 — Ingestion Snowflake via Snowpipe - **Snowpipe** détecte automatiquement les nouveaux fichiers Parquet dans S3 et les ingère dans une table Snowflake raw_conversations. - Modélisation aval en couches (architecture

médaille simplifiée) : raw_* → clean_* → analytics_* via tâches Snowflake (Streams + Tasks).

Étape 4 — Dashboards métier - Connexion Streamlit (ou alternative comme Metabase / Lightdash) à Snowflake. - Cinq KPIs principaux (F18) : - Volume de requêtes par jour / heure - Taux de satisfaction (feedback (+) / (-)) - Latence p50 / p95 / p99 - Coût LLM par jour et par requête - Taux d'utilisation de la recherche web (signal de couverture insuffisante du corpus)

Étape 5 — Insights produit - L'équipe produit (persona P4) identifie les requêtes mal traitées, les villes/catégories peu couvertes, les pics d'usage. - Ces insights alimentent le backlog des sprints suivants (boucle d'amélioration continue).

4.4.5 Synthèse des composants par catégorie

Pour faciliter la lecture du futur diagramme, voici la correspondance entre composants et catégories visuelles recommandées (couleurs / zones du diagramme) :

Catégorie	Composants	Code couleur recommandé
Utilisateur / Edge	Route 53, ACM, ALB	Bleu clair
Compute applicatif	ECS Fargate, ECR	Orange AWS
Données opérationnelles	DynamoDB, S3, Qdrant Cloud	Vert
IA externe	Mistral, Brave, API BAN	Violet
Sécurité et config	Secrets Manager, IAM, VPC	Rouge
Observabilité	CloudWatch, Langfuse	Jaune
Analytique	Snowflake, Snowpipe	Bleu foncé
IaC / CI/CD	Terraform, GitHub Actions	Gris (transverse)

Sous-section 4.5 — Stratégies transverses

4.5 Stratégies transverses

L'architecture présentée en 4.4 décrit **quoi** est déployé et **où**. Cette sous-section décrit **comment** elle est construite, maintenue, surveillée et sécurisée. Quatre stratégies transverses sont définies : déploiement, modularité, monitoring, sécurité.

4.5.1 Stratégie de déploiement

Principes directeurs Le déploiement du MVP suit trois principes structurants :

1. **Reproductibilité par Infrastructure-as-Code** : toute l'infrastructure AWS est décrite en Terraform versionné dans le repo. Aucune action manuelle dans la console AWS n'est tolérée hors investigations exceptionnelles.

2. **Automatisation par CI/CD** : tout commit sur main déclenche un pipeline qui build, teste, et déploie sur staging. La promotion en production est manuelle pour le MVP (validation explicite humaine).
3. **Environnements séparés** : staging et production sont strictement isolés (comptes AWS distincts, ou a minima VPC + IAM distincts), avec des données et des budgets indépendants.

Environnements

Environnement	Cible	Données	Déploiement
Local	Développement	Données mock ou Qdrant local	docker compose up
Staging	Validation pré-prod	Sous-ensemble réel anonymisé	Automatique sur merge main
Production	Utilisateurs réels	Données complètes	Manuel après validation staging

Infrastructure as Code — Terraform Le code Terraform est organisé en **modules réutilisables** pour favoriser la lisibilité, la testabilité et la réutilisation :

```
infra/ └─ modules/ └─ networking/ # VPC, subnets, security groups └─ compute/
# ECS Fargate, ECR, ALB └─ data/ # DynamoDB, S3 buckets └─ secrets/ #
Secrets Manager, IAM roles └─ observability/ # CloudWatch log groups, alarms └─
environments/ └─ staging/ # main.tf, variables.tfvars └─ production/ # main.tf,
variables.tfvars └─ README.md
```

État Terraform stocké dans un bucket S3 dédié `puls-events-tfstate` avec versioning activé et verrouillage via DynamoDB (`tflock`). Cette pratique standard évite les conflits en cas de déploiements concurrents.

Pipeline CI/CD — GitHub Actions Le pipeline comprend cinq étapes :

1. **Lint et tests unitaires** (ruff + pytest) sur chaque pull request.
2. **Build de l'image Docker** multi-stage (image finale < 500 MB) sur merge main.
3. **Push vers ECR** avec tag `sha-{git_sha}` et `latest`.
4. **Déploiement automatique sur staging** via `aws ecs update-service` avec `wait` sur le steady state.
5. **Smoke tests post-déploiement** (5 requêtes types via curl, validation des codes HTTP et de la latence).

Le déploiement production est déclenché **manuellement** depuis GitHub Actions par un membre habilité (`workflow_dispatch`).

Stratégie de release Pour le MVP, la stratégie retenue est **rolling update** (remplacement progressif des conteneurs ECS) avec un minimum de 50% de capacité disponible pendant le déploiement. Cette stratégie offre un bon compromis simplicité/disponibilité pour une équipe restreinte.

Une stratégie **blue/green** ou **canary** sera évaluée post-MVP si le volume de trafic le justifie.

4.5.2 Stratégie de modularité

Principe : la réversibilité par abstraction L'écosystème IA évolue à un rythme tel qu'aucun choix de composant ne peut être considéré comme définitif sur un horizon > 12 mois. La stratégie de modularité du MVP repose donc sur le principe de **réversibilité architecturale** : chaque dépendance externe est isolée derrière une **interface abstraite**, ce qui permet de la remplacer sans toucher au code métier.

Ce principe s'inspire de l'**Architecture Hexagonale** (Ports & Adapters, Alistair Cockburn, 2005), redevenue particulièrement pertinente avec la volatilité des fournisseurs LLM en 2025-2026.

Couches de l'application L'application est organisée en quatre couches logiques internes, indépendamment de l'infrastructure :

Couche	Responsabilité	Exemples
Domain	Logique métier pure, sans dépendance externe	Modèles Event, Conversation, règles de scoring géo
Use Cases	Orchestration des cas d'usage	AnswerCulturalQuery, RefreshEventCatalog
Adapters	Implémentations des interfaces externes	MistralLLMProvider, QdrantVectorStore, BraveSearchClient, DynamoDBCheckpointner
Infrastructure	Configuration, démarrage, injection	FastAPI, Streamlit, points d'entrée Docker

Le **code métier (Domain + Use Cases) ne dépend jamais directement d'un fournisseur** : il manipule des interfaces définies dans la couche Domain. Les adaptateurs concrets sont injectés au démarrage.

Interfaces abstraites principales

Interface	Implémentation MVP	Alternatives documentées
LLMProvider	MistralProvider	OpenAIProvider, AnthropicProvider, OllamaProvider (local)
EmbeddingProvider	MistralEmbedProvider	OpenAIEmbedProvider, HuggingFaceEmbedProvider

Interface	Implémentation MVP	Alternatives documentées
VectorStore	QdrantVectorStore	PineconeVectorStore, PGVectorStore
ConversationStore	DynamoDBCheckpointter	PostgresCheckpointter, RedisCheckpointter
WebSearchProvider	BraveSearchProvider	TavilyProvider, ExaProvider
Geocoder	BANGeocoder	NominatimGeocoder, GoogleMapsGeocoder
Observability	LangfuseTracer	PhoenixTracer, LangSmithTracer

Bénéfices pratiques Cette discipline apporte trois bénéfices mesurables :

1. **Mitigation du risque R9** (indisponibilité prolongée d'une dépendance critique) : un changement de fournisseur LLM prend quelques heures de configuration, pas des semaines de réécriture.
2. **Testabilité** : les use cases métier peuvent être testés avec des implémentations factices (mocks) sans appel API réel, ce qui accélère la suite de tests et réduit les coûts en CI.
3. **Veille technologique facilitée** : tester un nouveau fournisseur (par exemple un modèle local Mistral hébergé sur Ollama) ne nécessite qu'une nouvelle implémentation d'adapter.

Note pour ton oral : si on te demande « *comment garantissez-vous que votre architecture résistera aux évolutions du marché IA ?* », ta réponse est : « *Par une stratégie de réversibilité explicite. Chaque dépendance externe est derrière une interface, ce qui me permet de changer de fournisseur sans toucher au code métier. C'est une discipline d'architecture hexagonale appliquée à l'écosystème LLM.* »

4.5.3 Stratégie de monitoring

Les trois niveaux de monitoring Conformément à la posture définie en 4.3.1, le monitoring du MVP couvre trois niveaux complémentaires, chacun avec ses outils et ses indicateurs :

Niveau	Outil	Question répondue
Infrastructure	CloudWatch	L'infrastructure tient-elle la charge ?
Application LLM	Langfuse	Le pipeline RAG fonctionne-t-il correctement ?
Métier	Snowflake + dashboard	Les utilisateurs trouvent-ils de la valeur ?

SLIs et SLOs définis pour le MVP Les **SLIs** (Service Level Indicators) sont les mesures techniques, les **SLOs** (Service Level Objectives) sont les objectifs internes associés. Aucun SLA contractuel n'est défini en MVP (pas d'engagement client formel à ce stade).

Niveau	SLI	SLO MVP	Outil de mesure
Infra	Disponibilité du endpoint / chat	$\geq 99\%$ sur 30 jours glissants	CloudWatch + healthcheck ALB
Infra	Latence p95 du endpoint / chat	≤ 5 secondes	CloudWatch métriques ALB
Infra	Taux d'erreur 5xx	$\leq 1\%$ des requêtes	CloudWatch
App LLM	Taux de réponse jugée pertinente (LLM-as-a-judge)	$\geq 80\%$ sur jeu d'évaluation hebdomadaire	Langfuse + RAGAS
App LLM	Coût LLM moyen par conversation	$\leq 0,05$ €	Langfuse
App LLM	Taux d'usage de la recherche web (signal de couverture)	$\leq 25\%$ des requêtes	Langfuse
Métier	Taux de satisfaction utilisateur ((+) / total feedback)	$\geq 75\%$	Langfuse → Snowflake
Métier	Taux de retour utilisateur à 7 jours	$\geq 30\%$ post-lancement	Snowflake (logs sessions)

Alerting Trois types d'alertes sont configurées dans CloudWatch et Langfuse :

- **Alertes critiques** (notification immédiate par email) : disponibilité $< 99\%$, taux d'erreur $> 5\%$, coût LLM journalier $> 2 \times$ la moyenne sur 7 jours.
- **Alertes d'avertissement** (notification batch quotidienne) : latence p95 $> 5s$ sur 1h, taux de satisfaction $< 70\%$ sur 24h, taux d'usage web $> 30\%$.
- **Alertes de drift** (revue hebdomadaire en sprint review) : baisse $> 10\%$ du score de pertinence sur le jeu d'évaluation hebdomadaire — déclenche une investigation R10.

Boucle d'amélioration continue Le monitoring n'a de valeur que s'il alimente une **boucle de décision**. Le rituel suivant est mis en place :

- **Quotidien** : revue rapide des alertes critiques par l'astreinte (rotation entre Tech Lead et Data Engineer).
- **Hebdomadaire** : revue Langfuse en sprint review (top 10 des conversations mal notées, top 10 des conversations les plus coûteuses, drift éventuel).

- **Mensuel** : revue Snowflake en comité produit (KPIs métier, identification des thématiques sous-couvertes, recommandations pour le backlog).
-

4.5.4 Stratégie de sécurité

Approche défense en profondeur La sécurité du MVP repose sur le principe de **défense en profondeur** : plusieurs couches de protection indépendantes, de sorte que la défaillance d'une couche ne compromette pas le système entier.

Sécurité réseau

- **VPC privé** avec sous-réseaux publics (ALB uniquement) et privés (ECS Fargate, DynamoDB endpoints).
- **Security Groups** restrictifs : ECS accepte uniquement l'ALB sur le port 8000, sortie vers Internet limitée aux domaines explicitement nécessaires (Mistral, Brave, Qdrant, Langfuse).
- **VPC Endpoints** pour DynamoDB et S3 : le trafic ne sort pas du réseau AWS, réduisant la surface d'attaque et les coûts NAT Gateway.
- **TLS 1.2+ obligatoire** pour tout le trafic externe, certificats gérés par ACM avec renouvellement automatique.

Gestion des identités et des accès (IAM) Le principe de **moindre privilège** est appliqué strictement :

- Un rôle IAM **par composant** (ECS task role, ECS execution role, ingestion Lambda role, Snowpipe integration role).
- Aucun rôle ne dispose de permissions `*:*` ou `aws:*`. Chaque permission est scope sur une ressource précise (ARN explicite).
- Pas d'utilisateur IAM long-lived pour le pipeline CI/CD : utilisation de **OIDC GitHub Actions** → **AWS** avec rôle assumé temporairement.

Gestion des secrets

- Toutes les clés API (Mistral, Brave, Qdrant, Snowflake, Langfuse) sont stockées dans **AWS Secrets Manager**, chiffrées au repos avec KMS.
- Les conteneurs ECS récupèrent les secrets à l'exécution via les **task definitions secrets** (jamais d'injection en variable d'environnement claire).
- **Rotation programmée** des clés API critiques tous les 90 jours (manuelle pour les fournisseurs ne supportant pas la rotation automatique).
- Aucun secret n'est jamais versionné dans Git (vérification automatique via git-secrets ou équivalent en pre-commit).

Conformité RGPD La fonctionnalité F21 (conformité RGPD) implique plusieurs mesures concrètes :

- **Consentement explicite** lors de la première interaction utilisateur, avec présentation claire des finalités de collecte (mémoire conversationnelle, amélioration produit).

- **Anonymisation des logs analytiques** : les session_id sont hashés (SHA-256 avec salt) avant export vers Snowflake. Aucune donnée nominative dans le data warehouse analytique.
- **TTL automatique sur DynamoDB** : 30 jours sur les états conversationnels, conformément au principe de minimisation de durée de conservation.
- **Droit à l'effacement** : endpoint API /user/delete permettant la suppression complète d'une session (DynamoDB + logs CloudWatch + traces Langfuse).
- **Localisation des données** : région AWS eu-west-3 (Paris), Mistral hébergé en UE, Langfuse Cloud avec option UE.
- **Registre des traitements** documenté et tenu à jour par le DPO en lien avec le persona équipe produit.

Protection contre les abus L'exposition publique d'un chatbot LLM crée un risque d'abus (prompt injection, scraping, dépassement de coûts). Trois mesures sont mises en place :

- **Rate limiting** au niveau ALB (par IP et par session) : 10 requêtes par minute par session, 100 par jour par IP.
- **Quota de coût par utilisateur** : maximum 1 € de coût LLM cumulé par jour par session, blocage soft au-delà.
- **Filtrage anti-injection** basique : détection des patterns classiques de prompt injection en pré-traitement de la requête utilisateur. Pour le MVP, approche défensive sans perfectionnisme — un audit dédié sera mené post-MVP si le volume d'usage le justifie.

Plan de réponse à incident Cinq scénarios d'incident sont documentés en runbook (F25) :

Scénario	Détection	Réponse
Indisponibilité Mistral API	Alerting Langfuse + CloudWatch	Bascule vers fournisseur secondaire via reconfiguration adapter
Dépassement de coût LLM	Alerte Langfuse coût journalier	Blocage temporaire des sessions non authentifiées + investigation
Fuite de clé API détectée	Audit logs + alertes externes	Rotation immédiate du secret + invalidation côté fournisseur
Dégradation qualité (drift)	Évaluation hebdomadaire	Investigation Langfuse traces + rollback déploiement si lié
Demande RGPD (effacement)	Email DPO ou formulaire	Exécution endpoint /user/delete + confirmation utilisateur

4.5.5 Stratégie de recette et de validation

Le déploiement, le monitoring et la sécurité garantissent que le produit *fonctionne* ; la recette et la validation garantissent qu'il est *accepté*. Cette stratégie définit comment, à chaque jalon, le commanditaire (équipe produit Puls-Events, en posture de maîtrise d'ouvrage) reconnaît formellement que le livrable est conforme et exploitable avant de passer à l'étape suivante.

Vérification et validation : deux questions distinctes La démarche repose sur la distinction classique de l'ingénierie logicielle :

- **Vérification** — « *a-t-on construit le produit correctement?* » Le livrable est conforme à ses spécifications techniques (tests unitaires, tests d'intégration, revue de code, conformité au contrat d'interface).
- **Validation** — « *a-t-on construit le bon produit?* » Le livrable répond au besoin réel des utilisateurs et des personas définis en Section 1, dans leurs conditions d'usage.

Un MVP peut être vérifié sans être validé : un chatbot peut respecter sa spécification tout en produisant des recommandations que les utilisateurs jugent peu pertinentes. La recette est le moment où ces deux contrôles se rejoignent et s'officialisent.

Les trois niveaux de recette

Niveau	Objet	Exemples concrets dans ce MVP
Recette technique	Le produit est déployable, stable, performant et sécurisé	Tests de charge Locust 100 utilisateurs (F20), respect des SLOs (latence, disponibilité), audit IAM et secrets, pipeline CI/CD vert
Recette fonctionnelle	Chaque fonctionnalité fait ce qui était spécifié	Validation des Must-Have du backlog : mémoire conversationnelle, filtrage géographique, routing RAG/Web, affichage des sources
Recette utilisateur (métier)	Les utilisateurs cibles confirment l'exploitabilité réelle	Évaluation étendue sur 50 questions annotées (F19), retours via le bouton feedback (F15), revue par l'équipe produit et marketing

Deux jalons de validation normés : VABF et VSR Pour un commanditaire public ou exigeant, la validation se découpe en deux paliers contractuels, adaptés ici à

l’horizon du MVP :

- **VABF — Vérification d’Aptitude au Bon Fonctionnement** : prononcée au jalon **J5 (Production)**, sur l’environnement de pré-production. Elle atteste que le MVP est apte à être mis en service. C’est la recette « à chaud ».
- **VSR — Vérification de Service Régulier** : prononcée au jalon **J6 (Bilan)**, après deux semaines d’exploitation réelle (sprint 6). Elle confirme la stabilité du service dans la durée, sur la base des observations Langfuse et CloudWatch. C’est la recette « à froid » qui clôt définitivement la garantie de conformité.

Critères d’acceptation La recette n’est prononcée que si les critères mesurables suivants sont atteints — ils transforment les SLOs et la qualité RAG en seuils de décision binaires :

Dimension	Critère d’acceptation (seuil)	Source de mesure
Pertinence RAG	Score moyen $\geq 4/5$ sur les 50 questions, 0 % de réponse vide	Jeu d’évaluation F19
Performance	Latence P95 conforme au SLO défini, sous 100 utilisateurs simultanés	Tests de charge Locust (F20)
Disponibilité	Aucun incident bloquant non résolu pendant la VSR	Monitoring Langfuse + CloudWatch
Conformité RGPD	Endpoint d’effacement fonctionnel, validation DPO obtenue	Recette sécurité (F21)
Fonctionnel	100 % des fonctionnalités Must-Have recettées sans réserve bloquante	Backlog MoSCoW

Gestion des réserves et procès-verbal À l’issue de chaque recette, un **procès-verbal (PV)** est signé. Il peut être assorti de **réserves**, classées par criticité :

- **Réserve bloquante** : empêche la mise en service ; correction obligatoire avant prononciation de la recette.
- **Réserve majeure** : acceptée sous condition de correction dans un délai convenu (typiquement avant la VSR).
- **Réserve mineure** : consignée, traitée dans la roadmap V1 sans bloquer la production.

Cette graduation évite l’écueil du « tout ou rien » : le MVP peut entrer en service avec des réserves mineures documentées, dans l’esprit même d’un *Minimum Viable Product*.

Répartition des responsabilités

Acteur	Rôle dans la recette
Maîtrise d'ouvrage (MOA) — équipe produit	Définit les critères d'acceptation, exécute la recette utilisateur, prononce et signe le PV
Maîtrise d'œuvre (MOE) — Data Engineer	Prépare les environnements et jeux de tests, conduit les recettes technique et fonctionnelle, lève les réserves
DPO	Valide la conformité RGPD comme préalable à la VABF
Tech Lead	Arbitre la criticité des réserves en cas de désaccord

Cette formalisation ancre le MVP dans une démarche d'ingénierie maîtrisée : la mise en production n'est pas un événement technique isolé mais une **décision tracée, mesurée et partagée** entre les parties prenantes.

Section 5 : Estimation des coûts BUILD et OPEX

Cette section présente l'estimation financière du projet, structurée en quatre temps : (1) hypothèses de charge servant de base aux calculs, (2) coûts BUILD (investissement one-shot de développement), (3) coûts OPEX (exploitation récurrente) selon trois scénarios de charge, (4) leviers d'optimisation budgétaire.

Distinction structurante : le BUILD est un investissement one-shot (CAPEX comptable) qui crée le produit. L'OPEX est le coût récurrent mensuel ou annuel nécessaire pour maintenir le produit en vie. Cette distinction est attendue par toute direction financière qui examinera ce dossier.

5.1 Hypothèses de charge — trois scénarios

Pour rendre les estimations OPEX significatives, trois scénarios de charge sont définis. Ils correspondent à des **paliers d'adoption plausibles** sur les 12 mois suivant le lancement du MVP.

Scénario	Utilisateurs uniques mensuels	Sessions / jour	Requêtes / jour	Hypothèse
S1 — Démarrage	500	50	150	Phase post-lancement, communication restreinte, tests early adopters

Scénario	Utilisateurs uniques mensuels	Sessions / jour	Requêtes / jour	Hypothèse
S2 — Croissance	5 000	500	1 500	Communication active, bouche-à-oreille, presse spécialisée
S3 — Succès	50 000	5 000	15 000	Couverture médiatique, intégration partenaires, viralité

Hypothèses unitaires retenues (cohérentes avec l'évaluation du POC rag-events - 2026) : - 3 requêtes en moyenne par session conversationnelle - 1 000 tokens d'input et 500 tokens de génération par requête - 1 appel d'embedding par requête (~50 tokens) - 25% des requêtes déclenchent une recherche web Brave (F12) - 10% des requêtes nécessitent une réécriture + nouveau retrieval

Ces hypothèses constituent la base des calculs LLM et infra qui suivent. Elles seront **réajustées trimestriellement** sur la base des données réelles observées dans Langfuse et Snowflake.

5.2 Coûts BUILD — investissement initial sur 12 semaines

5.2.1 Coûts humains (poste principal)

L'équipe est dimensionnée selon la composition définie en section 1.8. Les coûts sont exprimés en **TJM marché français 2026** (taux journalier moyen consultant/freelance), soit l'équivalent économique qu'aurait à dépenser Puls-Events si elle faisait appel à des prestataires externes. Cette unité est privilégiée pour sa transparence et son comparabilité.

Rôle	TJM 2026	Charge (jours)	Coût total
Data Engineer (pilote technique)	600 €	60 j (100% × 12 semaines)	36 000 €
Tech Lead / Responsable technique	800 €	12 j (20%)	9 600 €
Product Owner	700 €	18 j (30%)	12 600 €
UX/UI Designer	650 €	10 j (50% × 4 semaines)	6 500 €
DevOps / SRE (appui)	750 €	6 j (10%)	4 500 €
Data Analyst	600 €	5 j (à partir S8)	3 000 €
DPO / Juridique	900 €	4 j (ponctuel)	3 600 €
Marketing / Growth	600 €	5 j (à partir S10)	3 000 €

Rôle	TJM 2026	Charge (jours)	Coût total
Sous-total humain		120 j-h	78 800 €

5.2.2 Coûts infrastructure et outillage (setup initial)

Poste	Détail	Coût BUILD
AWS — environnement staging	Free Tier sur 12 semaines (ECS, S3, DynamoDB, CloudWatch)	0 €
AWS — provisioning Terraform	Configuration initiale + tests	0 € (couvert par temps Data Engineer)
Qdrant Cloud	Free Tier 1 GB	0 €
Mistral AI	Free Tier + crédits initiaux pour vectorisation (5000 events × 1024-dim)	~20 €
Brave Search API	Free Tier 2 000 req/mois — tests inclus	0 €
Langfuse Cloud	Free Tier 50k observations	0 €
Snowflake	Free Trial \$400 (~370 €)	0 €
Domaine puls-events.fr	Achat + 1 an	15 €
Certificat SSL	AWS Certificate Manager (gratuit)	0 €
GitHub Actions	Gratuit jusqu'à 2 000 min/mois (repo privé)	0 €
Outils de design (Genspark, Figma)	Licences ponctuelles 12 semaines	50 €
Sous-total outillage		85 €

5.2.3 Synthèse BUILD

Catégorie	Montant
Coûts humains	78 800 €
Coûts infrastructure et outillage	85 €
Total BUILD	78 885 €

Lecture du tableau : le coût humain représente **99,9% du BUILD**. Ce ratio est typique d'un MVP IA en 2026 : grâce aux services managés et aux Free

Tier généreux, l'infrastructure de développement est quasi-gratuite. Le **vrai investissement** d'un MVP IA, ce sont les **personnes** qui le conçoivent.

5.3 Coûts OPEX — exploitation récurrente par scénario

Les coûts OPEX sont calculés **par mois** pour chacun des trois scénarios définis en 5.1. Toutes les valeurs sont en euros, taxes comprises lorsque applicable.

5.3.1 Calcul des coûts LLM (Mistral AI)

Tarifs Mistral 2026 retenus : - mistral-small : 0,14 € / M tokens input, 0,56 € / M tokens output (conversion USD→EUR) - mistral-embed : 0,10 € / M tokens (tier payant au-delà du free)

Calculs détaillés en fonction des hypothèses unitaires 5.1 :

Élément	S1 (Démarrage)	S2 (Croissance)	S3 (Succès)
Requêtes / mois	4 500	45 000	450 000
Tokens input LLM (M)	4,5	45	450
Tokens output LLM (M)	2,3	22,5	225
Tokens embeddings (M)	0,2	2,3	22,5
Coût LLM input	0,63 €	6,30 €	63 €
Coût LLM output	1,26 €	12,60 €	126 €
Coût embeddings	0,02 €	0,23 €	2,25 €
Coût Mistral mensuel	~2 €	~20 €	~190 €

5.3.2 Calcul des coûts AWS (eu-west-3 Paris)

Hypothèses retenues pour le dimensionnement ECS Fargate : - S1 : 1 tâche permanente × 0,5 vCPU × 1 GB RAM (24/7) - S2 : 2 tâches × 0,5 vCPU × 1 GB RAM (24/7, auto-scaling) - S3 : 4 tâches × 1 vCPU × 2 GB RAM (24/7, auto-scaling)

Tarifs AWS eu-west-3 2026 : - Fargate : 0,04 € / vCPU-heure + 0,0045 € / GB-RAM-heure - DynamoDB on-demand : 1,25 € / M lectures, 6,25 € / M écritures, 0,25 € / GB-mois stockage - S3 Standard : 0,023 € / GB-mois + 0,005 € / 1000 requêtes - CloudWatch : 0,50 € / GB logs ingérés + alarmes

Poste AWS	S1	S2	S3
ECS Fargate compute	18 €	36 €	145 €
ALB (load balancer)	18 €	18 €	22 €
DynamoDB (R/W + stockage)	1 €	8 €	60 €
S3 (3 buckets, ~10 GB)	1 €	2 €	10 €
CloudWatch (logs + alarmes)	2 €	8 €	40 €

Poste AWS	S1	S2	S3
Secrets Manager (~10 secrets)	4 €	4 €	4 €
Route 53 (DNS + requêtes)	1 €	1 €	2 €
Data transfer (sortie internet)	2 €	10 €	80 €
Sous-total AWS	~47 €	~87 €	~363 €

5.3.3 Calcul des coûts services tiers

Service	S1	S2	S3	Justification
Qdrant Cloud	0 €	0 €	~25 €	Free Tier suffit jusqu'à S3 (passage au Standard à partir de ~30k events)
Brave Search API	0 €	~15 €	~150 €	Free Tier 2000 req/mois suffit S1, payant ensuite
Langfuse Cloud	0 €	0 €	~50 €	Free Tier 50k obs/mois suffit S1+S2, plan Pro requis ou bascule self-host en S3
Snowflake	0 € (Free Trial) puis ~30 €	~50 €	~120 €	Free Trial 30j puis ~30€/mois minimum (1 warehouse XS occasionnel)
Sous-total tiers	~30 €	~65 €	~345 €	

5.3.4 Synthèse OPEX mensuelle

Catégorie	S1 (Démarrage)	S2 (Croissance)	S3 (Succès)
LLM (Mistral)	2 €	20 €	190 €
AWS infrastructure	47 €	87 €	363 €
Services tiers	30 €	65 €	345 €

Catégorie	S1 (Démarrage)	S2 (Croissance)	S3 (Succès)
Total OPEX / mois	~79 €	~172 €	~898 €
Total OPEX / an	~950 €	~2 060 €	~10 770 €
Coût par utilisateur actif / mois	~0,16 €	~0,03 €	~0,02 €

Lecture économique : le coût par utilisateur **diminue fortement avec la charge** (de 0,16 € à 0,02 €). C'est l'effet d'échelle classique d'une architecture cloud-native : les coûts fixes (ALB, Secrets Manager, Qdrant minimum, Snowflake minimum) sont absorbés par un volume croissant. Ce ratio est un argument business fort pour défendre la scalabilité du modèle.

5.4 Synthèse globale sur 12 mois

Projection cumulée du coût total sur la première année post-MVP, en combinant l'investissement BUILD initial et l'OPEX mensuel.

Scénario réaliste — trajectoire mixte

Hypothèse de trajectoire réaliste : 3 mois en S1 (démarrage), 6 mois en S2 (croissance), 3 mois en S3 (succès).

Période	Mois	OPEX cumulé
Mois 1-3	S1 (Démarrage)	3 × 79 € = 237 €
Mois 4-9	S2 (Croissance)	6 × 172 € = 1 032 €
Mois 10-12	S3 (Succès)	3 × 898 € = 2 694 €
OPEX 12 premiers mois		3 963 €
BUILD initial		78 885 €
TOTAL 12 mois		~82 850 €

Lecture stratégique

Cette projection met en évidence trois faits structurants pour une direction financière :

1. **Le BUILD pèse 95% du coût total sur la première année**, ce qui est caractéristique d'un MVP. À mesure que le produit dure dans le temps, ce ratio s'inverse au profit de l'OPEX.
2. **Le seuil d'amortissement** est déterminé par la valeur générée (ARR — annual recurring revenue, abonnements, partenariats), non par les coûts seuls. Le rapport ne traite pas du revenu mais ce point doit être posé pour le COO.
3. **L'OPEX reste sous 1 000 €/mois même au scénario succès**, ce qui constitue un argument économique fort : la structure technique est conçue pour absorber 100× plus d'utilisateurs sans changement majeur d'architecture.

5.5 Leviers d'optimisation budgétaire

Cinq leviers d'optimisation sont identifiés, classés par impact décroissant.

Levier 1 — Cache d'embeddings (impact LLM)

Les requêtes utilisateurs présentent une forte récurrence (mêmes villes, mêmes catégories). Un cache Redis (ElastiCache ou simple DynamoDB) sur les embeddings de requêtes peut **réduire de 30 à 50%** le coût Mistral Embed.

- Coût d'implémentation : ~2 jours-homme
- Économie au scénario S3 : ~70 € / an
- Rentabilité : neutre en MVP, devient intéressant à grande échelle

Levier 2 — Réservation de capacité Fargate (Compute Savings Plans)

Les AWS Compute Savings Plans permettent d'obtenir **jusqu'à 50% de réduction** sur Fargate en s'engageant sur 1 ou 3 ans. À évaluer post-MVP une fois la trajectoire de charge confirmée.

- Coût d'implémentation : 0
- Économie au scénario S3 : ~870 € / an
- Recommandation : **attendre 6 mois d'OPEX réel** avant d'engager

Levier 3 — Politique de rétention des logs

Réduire la rétention CloudWatch (de 30 à 7 jours par défaut) et appliquer une politique de cycle de vie S3 vers Glacier sur les logs anciens.

- Coût d'implémentation : ~0,5 jour-homme
- Économie au scénario S3 : ~120 € / an
- Recommandation : **à appliquer dès la mise en production**

Levier 4 — Bascule Langfuse self-hosted

À partir du moment où le volume dépasse 50k observations/mois (scénario S3), Langfuse Cloud devient payant. Bascule vers un self-hosting sur ECS Fargate (~15 €/mois d'infra) plutôt que le plan Pro (~70-100 €/mois).

- Coût d'implémentation : ~3 jours-homme
- Économie au scénario S3 : ~600 € / an
- Recommandation : **à déclencher au franchissement du seuil**

Levier 5 — Optimisation des prompts LLM

Réduction de 20% de la longueur des prompts par optimisation (suppression d'instructions redondantes, compression de contexte) réduit proportionnellement le coût token input.

- Coût d'implémentation : itératif, ~1 jour-homme par sprint
- Économie au scénario S3 : ~40 € / an + amélioration latence
- Recommandation : **discipline continue intégrée aux rétrospectives**

Synthèse leviers

Levier	Effort	Économie annuelle (S3)	Priorité
Cache embeddings	2 j-h	70 €	Faible (MVP) — Moyenne (V1)
Compute Savings Plans	0	870 €	Élevée à 6 mois
Rétention logs	0,5 j-h	120 €	Élevée immédiate
Langfuse self-host	3 j-h	600 €	Moyenne (déclenchée par se
Optimisation prompts	continu	40 €	Faible mais continue
Économies cumulables (S3)		~1 700 € / an	

L'application des leviers prioritaires permettrait de **réduire l'OPEX du scénario S3 d'environ 20%**, ramenant la charge annuelle de ~10 770 € à ~9 070 €.

5.6 Annexe : disposition Excel / CSV

Le détail des calculs ligne à ligne est disponible en annexe au format CSV importable Excel ou Google Sheets : annexes/couts_mvp.csv. Ce fichier contient l'ensemble des postes BUILD et OPEX, les hypothèses unitaires, et permet de recalculer dynamiquement les coûts en modifiant les paramètres d'entrée (volume de requêtes, TJM, tarifs fournisseurs).

Conclusion du rapport

Synthèse

Ce rapport a présenté la transformation conceptuelle d'un POC RAG validé , en MVP scalable cloud-native, livrable dans un horizon de 12 semaines pour un investissement BUILD estimé à 79 000 € .

L'architecture cible repose sur quatre principes structurants qui se renforcent mutuellement : - la **continuité technologique** avec le POC expérimental (LangGraph, Qdrant, Mistral repris de rag-events-2026), - la **réversibilité** des dépendances externes via une architecture hexagonale, - la **séparation** stricte entre données opérationnelles (AWS) et données analytiques (Snowflake), - et la **discipline d'observabilité** sur trois niveaux complémentaires (infrastructure, application LLM, métier).

La démarche projet adopte une méthodologie Scrumban hybride dimensionnée pour une équipe restreinte, structurée autour de six jalons Go/No-Go indépendants des sprints, et matérialisée par un macro backlog de 32 fonctionnalités priorisées MoSCoW et estimées en story points Fibonacci.

Apprentissages structurants

Au-delà des livrables techniques, cette mission m’a fait expérimenter une **bascule de posture professionnelle** qui constitue probablement l’apport le plus durable du projet. Là où la mission POC évaluait ma capacité à exécuter techniquement, celle-ci a mobilisé ma capacité à **décider, justifier, anticiper et communiquer**.

Trois apprentissages me semblent particulièrement structurants pour la suite de mon parcours.

Premier apprentissage : la rigueur de la **veille technologique** n’est pas un exercice académique mais un outil de pilotage. Comparer systématiquement les options, documenter ce qu’on écarte autant que ce qu’on retient, et challenger les suggestions reçues — y compris celles du responsable technique sur smolagents — sont des disciplines qui distinguent un référent technique d’un simple exécutant.

Deuxième apprentissage : l’**honnêteté méthodologique** est plus crédible que l’illusion de la maîtrise. Étiqueter les personas comme hypothétiques, assumer qu’aucun SLA contractuel n’est défini en MVP, documenter explicitement la surcharge volontaire du Must-Have (110 SP pour 100 SP de capacité) — ces choix de transparence renforcent la crédibilité globale du document plus qu’ils ne la fragilisent.

Troisième apprentissage : l’**architecture est un système cohérent**, pas une collection de bons choix indépendants. Le choix AWS, le choix Snowflake cloud-agnostic, le choix de l’architecture hexagonale, le choix Mistral OpenAI-compatible, le choix Brave Search plutôt que Tavily — tous ces choix individuellement défendables prennent leur sens parce qu’ils convergent vers une même propriété système : la **réversibilité**.

Perspectives post-MVP

Le présent rapport décrit un MVP, c’est-à-dire un point de départ viable, et non un produit fini. Trois axes d’évolution structurants sont déjà identifiés.

Sur le **plan fonctionnel**, le passage à la V1 intégrera les fonctionnalités classées “Won’t-Have MVP” : notifications push personnalisées, intégrations billetterie partenaires, recommandations proactives par clustering utilisateurs, et application mobile native.

Sur le **plan technique**, plusieurs optimisations sont déjà identifiées : reranking via cross-encoder pour améliorer la qualité top-K, bascule de Langfuse vers un hébergement self-hosted au franchissement du seuil de 50k observations mensuelles, évaluation d’une migration cloud vers Scaleway pour renforcer la souveraineté et réduire les coûts d’un facteur potentiel de 4 à 5.

Sur le **plan organisationnel**, le succès du MVP devra s’accompagner d’une montée en maturité du pilotage : transition vers du Scrum complet si l’équipe dépasse 5 développeurs, formalisation de SLAs contractuels avec les premiers partenaires institutionnels, et constitution d’un Data Team étendu pour exploiter pleinement les gisements analytiques constitués dans Snowflake.

Position personnelle

Au terme de ce projet, ma conviction est renforcée que le métier de **Data Engineer Full Stack** en 2026 n'est plus défini par une collection d'outils maîtrisés, mais par la **capacité à orchestrer des systèmes complexes** combinant données, intelligence artificielle, infrastructure cloud et coordination humaine. Le code reste un moyen, pas une fin ; la valeur se déplace progressivement vers la **qualité des décisions** prises en amont de l'écriture du code.

Cette mission m'a donné l'occasion de pratiquer cette posture dans des conditions réalistes. Elle conforte mon projet professionnel : porter, dans les organisations qui m'accueilleront, des produits data de bout en bout, en alliant rigueur technique, vision systémique et honnêteté méthodologique.

Annexes

A1. Portfolio professionnel

Mon portfolio professionnel complet est hébergé sur Notion, par choix délibéré de contrôle de l'exposition des données face aux outils de scraping automatisé. La source de vérité technique reste GitHub.

Lien portfolio Notion :

→ [Behram K. — Data Engineer Portfolio](#)

Le portfolio est structuré autour de **quatre piliers** cohérents :

- **Data Platform Foundations** — Spark, dbt, Airflow, streaming Kafka, Data Lake / Data Warehouse
- **Software Craftsmanship** — Python 3.12+, tests, CI, observabilité, ADR
- **AI / LLM Engineering** — RAG, systèmes agentiques, structured extraction, évaluation
- **Project Leadership** — gestion de projet data, veille, estimation, architecture documentée

Il référence **10+ projets publics** documentés (contexte métier, architecture, stack, résultats, learnings) avec lien vers le repository GitHub correspondant.

A2. Repositories GitHub clés

Repository	Description	Lien
puls-events-mvp-design	Design d'architecture et gestion de projet (présent rapport)	github.com/behramkorkut/puls-events-mvp-design
rag-events-2026	Chatbot RAG production-ready France entière (socle technique du MVP)	github.com/behramkorkut/rag-events-2026

Repository	Description	Lien
puls-events-rag	POC initial Île-de-France (point de départ de la mission)	github.com/behramkorkut/puls-events-rag
Profil GitHub complet	Vue d'ensemble des projets data engineering	github.com/behramkorkut

A3. Schémas d'architecture

A3.1 Vue V1 — Vue d'ensemble en couches

Le diagramme V1 généré via Genspark est inséré ci-dessous. Il représente l'architecture en 6 couches décrite en section 4.4.1.

Architecture cible — Vue V1

Vue d'ensemble en couches — Région AWS eu-west-3 (Paris)

A3.2 Vues V2, V3, V4 (description textuelle)

Les vues V2 (flux d'ingestion), V3 (flux conversationnel) et V4 (flux analytique) sont décrites textuellement en sections 4.4.2 à 4.4.4 du présent rapport. Une version visuelle de la vue V3 pourra être produite pour la soutenance.

A4. Fichiers de données

Les fichiers de données structurés du projet sont disponibles dans le dossier annexes/ du repository GitHub associé.

Fichier	Contenu	Référencé en section
annexes/gantt_mvp.csv	Planning détaillé sur 12 semaines (tâches, sprints, jalons)	Section 2.3
annexes/backlog_mvp.csv	Backlog complet 36 fonctionnalités MoSCoW + story points + risques	Section 3.6
annexes/couts_mvp.csv	Estimation BUILD + OPEX par scénario de charge	Section 5.6

Ces fichiers sont au format CSV importable dans Excel ou Google Sheets. Ils sont versionnés dans le repo et destinés à être mis à jour à chaque sprint review du projet réel.

A5. Journal de bord du projet

La démarche de conception a été documentée au fil de l'eau dans un journal de bord (JOURNALDEBORD.md) versionné dans le repository. Il consigne les décisions

structurantes, les arbitrages techniques, et les questions ouvertes session par session. Cette pratique, inspirée des **Architecture Decision Records (ADR)**, permet :

- la **traçabilité méthodologique** des choix retenus,
- la **reprise du contexte** en cas de pause ou de transmission,
- la **transparence** sur le processus de réflexion vs. le résultat final.

A6. Sources et références bibliographiques

A6.1 Veille marché et benchmarks 2026

- **AWS vs Azure vs Google Cloud 2026** — Tech Insider, 2026.
- **Comparing AWS, Azure, and GCP for Startups in 2026** — DigitalOcean Cloud Market Report, 2026.
- **Scaleway vs OVHcloud benchmark** — Callista Independent Benchmark, février 2026.
- **Best Vector Databases in 2026 : Complete Comparison Guide** — Encore Dev, 2026.
- **Agentic Search in 2026 : Benchmark 8 Search APIs for Agents** — AiMultiple, 2026 (score Brave Search : 14.89, position 1).
- **Comprehensive comparison of every AI agent framework in 2026** — Communauté LangChain, 2026.
- **Tavily Alternatives in 2026 (After the Nebius Acquisition)** — Unicodeveloper, 2026.

A6.2 Documentation technique officielle

- **LangGraph Documentation** — LangChain Inc., 2026.
- **Qdrant Cloud Documentation et Pricing** — Qdrant Solutions GmbH, 2026.
- **Mistral AI Documentation** — Pricing et SDK, 2026.
- **Langfuse Documentation** — LLM Observability, 2026.
- **Snowflake Documentation** — Snowpipe, Cortex AI, 2026.
- **AWS Documentation** — ECS Fargate, DynamoDB, IAM, Secrets Manager, 2026.
- **Brave Search API Documentation** — Brave Software, 2026.

A6.3 Fondements méthodologiques et architecturaux

- **Hexagonal Architecture (Ports & Adapters)** — Alistair Cockburn, 2005.
- **The C4 Model for Software Architecture** — Simon Brown, 2015.
- **Site Reliability Engineering** — Google SRE Book, 2016 (SLI / SLO / SLA).
- **Architecture Decision Records (ADR)** — Michael Nygard, 2011.
- **The Scrum Guide** — Schwaber & Sutherland, dernière édition.
- **Kanban : Successful Evolutionary Change for Your Technology Business** — David J. Anderson, 2010.

A6.4 Études sectorielles mobilisées

- **Pratiques culturelles des Français** — Ministère de la Culture, enquête 2023.
- **Baromètre du numérique** — ARCEP / CREDOC, édition 2024.
- **Études Médiamétrie** sur la consommation culturelle digitale.

A7. Outils utilisés pour la production du rapport

Outil	Usage
VSCode + Markdown	Rédaction du rapport
Genspark	Génération du diagramme d’architecture V1
GitHub	Versionning du rapport et des fichiers associés
Notion	Hébergement du portfolio professionnel
Claude (Anthropic)	Sparring partner méthodologique et pédagogique

Note de transparence : la conception de ce rapport a bénéficié d’un dialogue itératif avec un assistant IA (Claude d’Anthropic) en posture de sparring partner méthodologique. Chaque section a été validée puis intégrée par mes soins ; le contenu final reflète mes décisions et ma compréhension. Le journal de bord (A5) trace ce processus en toute transparence.

A8. Glossaire des acronymes

Acronyme	Signification
ADR	Architecture Decision Record
ALB	Application Load Balancer (AWS)
API	Application Programming Interface
BAN	Base Adresse Nationale (gouv.fr)
BI	Business Intelligence
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
DPO	Data Protection Officer
DWH	Data Warehouse
ECR	Elastic Container Registry (AWS)
ECS	Elastic Container Service (AWS)
ETP	Équivalent Temps Plein
IaC	Infrastructure as Code
IAM	Identity and Access Management (AWS)
LLM	Large Language Model

Acronyme	Signification
MCP	Model Context Protocol
MVP	Minimum Viable Product
NLP	Natural Language Processing
OPEX	Operating Expenditure (coûts d'exploitation récurrents)
BUILD / CAPEX	Capital Expenditure (investissement initial)
POC	Proof of Concept
RAG	Retrieval-Augmented Generation
RGPD	Règlement Général sur la Protection des Données
ROI	Return on Investment
SDK	Software Development Kit
SLA	Service Level Agreement
SLI	Service Level Indicator
SLO	Service Level Objective
SP	Story Points
SRE	Site Reliability Engineering
SSL/TLS	Secure Sockets Layer / Transport Layer Security
TJM	Taux Journalier Moyen
TTL	Time To Live
UC	Use Case
UI	User Interface
US	User Story
UX	User Experience
VPC	Virtual Private Cloud (AWS)
WIP	Work In Progress